02 线性表

- 链表
- 单链表
- 循环链表
- 双向链表
- 静态链表

02 线性表

- 链表

- 单链表

- 循环链表

- 双向链表

- 静态链表

链表(Linked List)

- 顺序表的优点
  - 相邻元素存储地址的间距是常数，因此存取任何元素速度快O(1)

- 顺序表的缺点
  - 插入或者删除元素时需要移动其它元素，代价太高O(n)
  - 静态分配存储空间时，难以确定存储空间大小，导致浪费或溢出
  - 动态分配存储空间时，能扩充存储空间，但时间开销较大

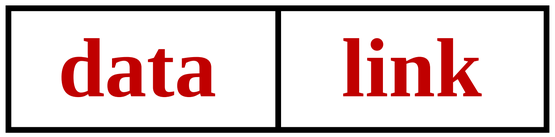
- 链表可以解决上述问题
  - 表元素可存在任何位置，每个元素都有下一个元素的地址(指针)
  - 适用于插入删除频繁、存储空间需求不定的问题场景

02 线性表

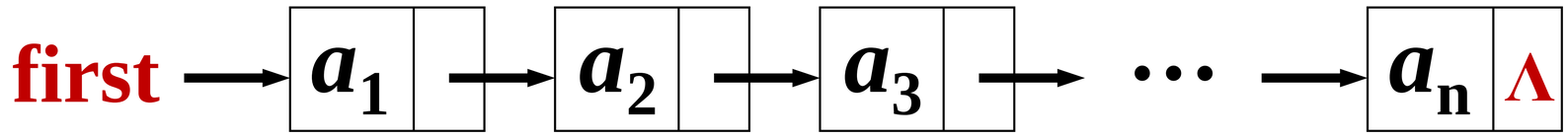
- 链表
- 单链表
- 循环链表
- 双向链表
- 静态链表

单链表(Singly Linked List)

- 每个元素是一个结点，由**数据域**和**指针/链域**组成



- 单链表的线性结构

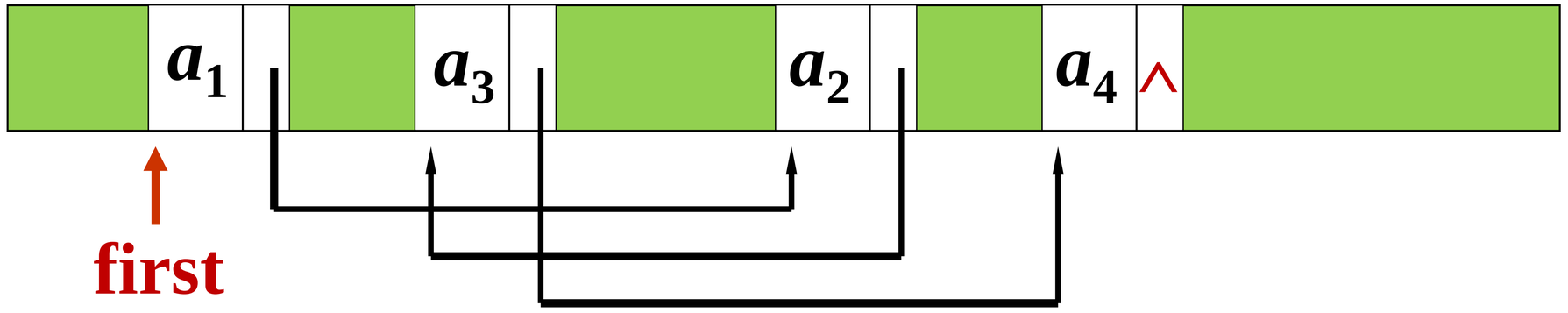


- 单链表的长度可以方便地进行扩充，结点的逻辑顺序与物理顺序可以不一致，即可以不连续存储

单链表的存储表示



(a) 可利用存储空间



(b) 经过一段运行后的单链表结构

单链表的类定义

```
template <class Type> class List;  // 单链表类的前向引用声明
template <class Type> class LinkNode {  // 单链表结点类定义
    friend class List <Type>;  // 单链表类作为友元类定义
    public:
        // 不带形参的构造函数
        LinkNode( ) { link = NULL; }
        // 带形参的构造函数
        LinkNode (const Type &value) { data = value; link = NULL; }
        ……
    private:
        Type data;  // 数据域
        LinkNode <Type> * link;  // 指针域
};
```

单链表的类定义

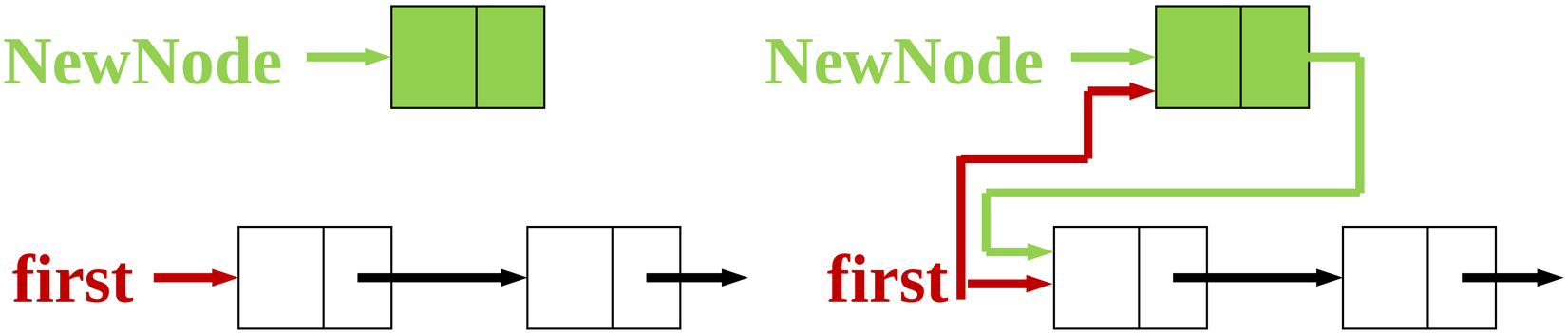
```
template <class Type> class List {  // 单链表类定义
    public:
        List(const Type &value) { first = new LinkNode <Type> (value); }
        ~List( );  // 析构函数
        LinkNode <Type> * Create_List(int n);  // 建立一个单链表
        void MakeEmpty( );  // 将单链表置为空表
        int Length( ) const;  // 计算单链表的长度
        LinkNode <Type> * Find(Type value);  // 搜索值为value的结点
        int Insert(Type value, int i);  // 在第i个位置插入值为value的结点
        int Delete(int i);  // 删除第i个结点
        ……
    private:
        LinkNode <Type> * first;  // 表头指针(还可以设置表尾指针)
};
```

建立单链表

```
template <class Type> LinkNode <Type> * Create_List(int n) {
    if (n==0)    return NULL;
    LinkNode *p, *q;
    p = first = new LinkNode <Type> (NULL);
    for (i = 1; i < n; i++) {
        cin>>p->data;
        q = new LinkNode <Type> (NULL);
        p->link = q;
        p = q;
    }
    cin>>p->data;
    p->link = NULL;
    return first;
}
```

单链表中的插入

- **第一种情况：在单链表最前端插入**



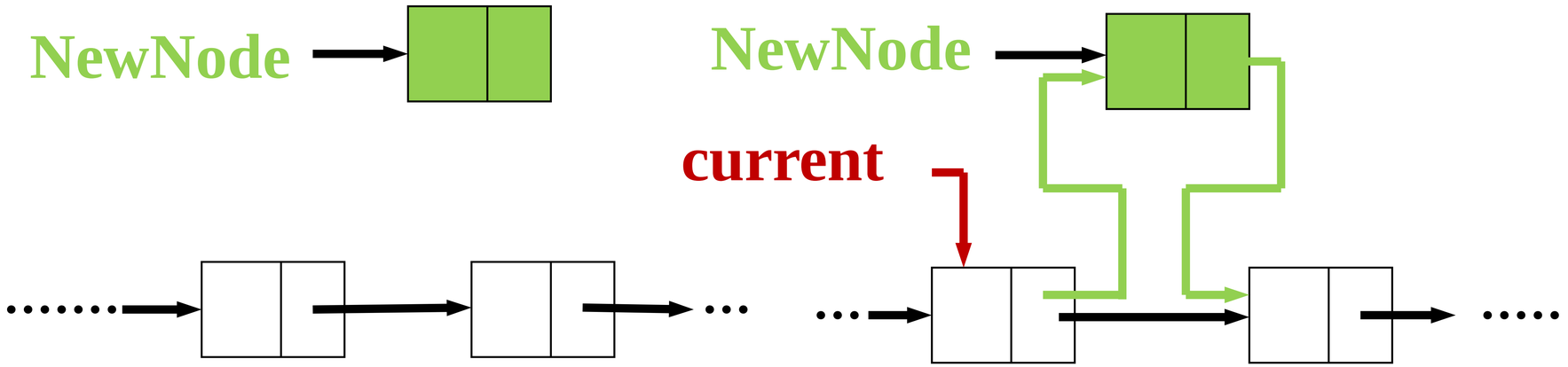
插入前　　插入后

NewNode->link = first;

first = NewNode;

单链表中的插入

- **第二种情况：在单链表中间插入**



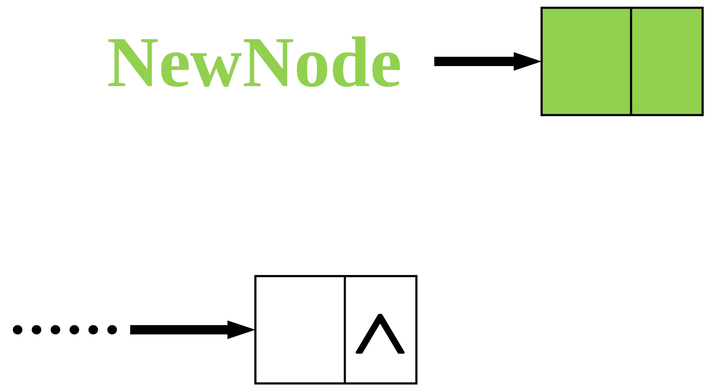
插入前　　插入后

NewNode->link = current->link;

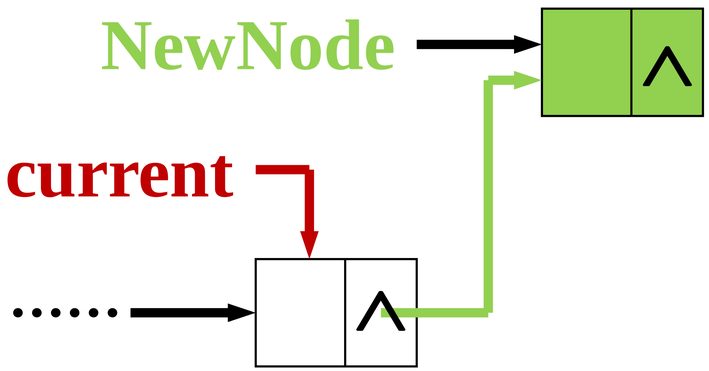
current->link = NewNode;

单链表中的插入

- **第三种情况：在单链表末尾插入**



插入前



插入后

```
NewNode->link = current->link;
current->link = NewNode;
```

单链表中的插入实现

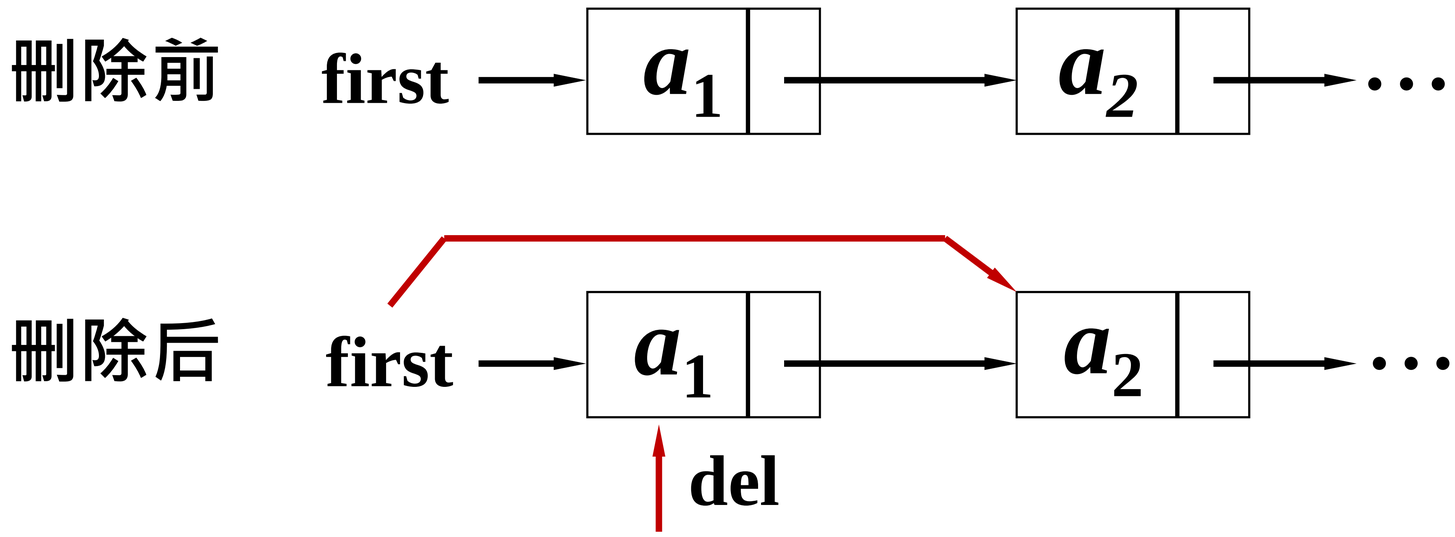
```
// 在第i个位置插入值为x的结点(i从1开始计数)
template <class Type> int List <Type> :: Insert(Type value, int i) {
    ListNode * current = first;
    for ( int k = 1; k < i - 1; k++ )  // 找到第i-1个结点
        if ( current == NULL ) break;
        else current = current->link;
if ( current == NULL && first != NULL ) {  // 非空表且链表太短
    cerr << " Invalid position for Insertion " <<endl;
    return 0;
}
```

单链表中的插入实现

```
ListNode * NewNode = new ListNode(value);  // 创建新结点
if ( first == NULL || i == 1 ) {  // 插在表前
    NewNode->link = first;
    first = NewNode;
} else {  // 插在表中或末尾
    NewNode->link = current->link;
    current->link = NewNode;
  }
  return 1;
}
```

单链表中的删除

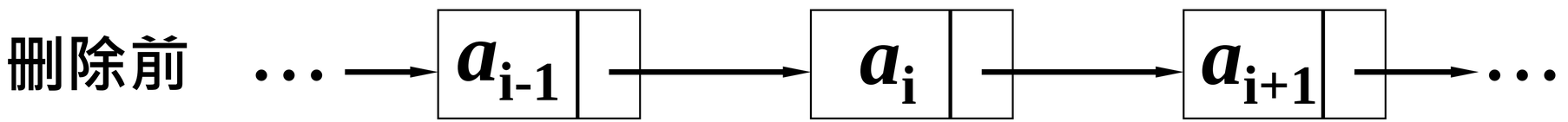
- **第一种情况**：删除表中第一个元素

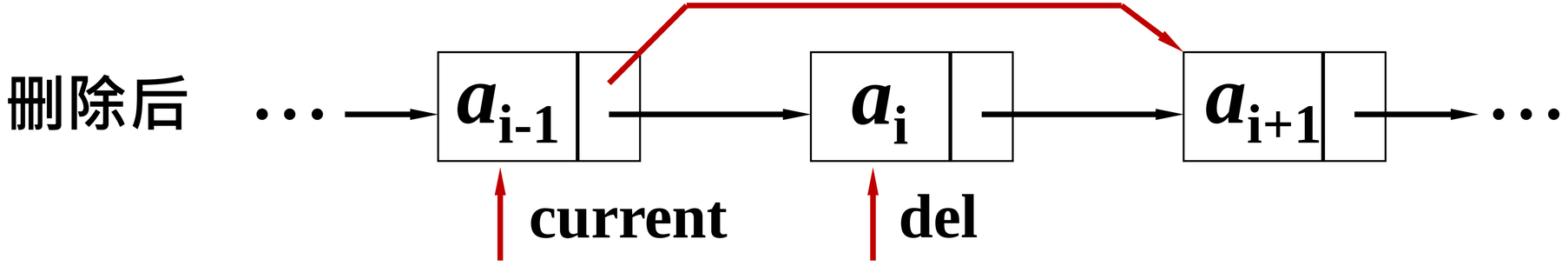



```
del = first;
first = first->link;
delete  del;
```

单链表中的删除

- **第二种情况**：删除表中或表尾元素






```
del = current->link;
current->link = del->link;
delete  del;
```

单链表中的删除实现

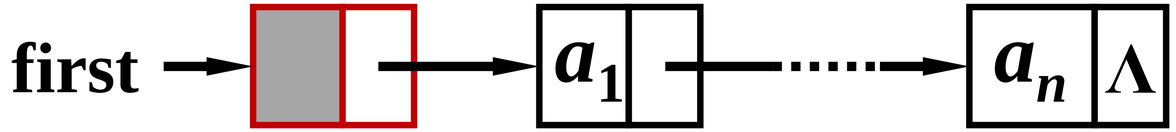
```
// 删除第i个结点(i从1开始计数)
template <class Type> int List <Type> :: Delete(int i) {
    ListNode * del, * current;
    current = first;
    for ( int k = 1; k < i - 1; k++ )  // 找到第i-1个结点
    if ( current == NULL ) break;
    else current = current->link;
if ( current == NULL || current->link == NULL ) {  //  链表太短
    cerr << " Invalid position for Insertion " <<endl;
    return 0;
}
```

单链表中的删除实现

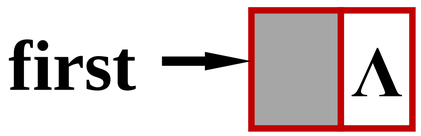
```
if ( i == 1 ) {  // 删除表中第1个结点
    del = first;
    first = first->link;
} else {
    del = current->link;
    current->link = del->link;
    }
    delete del;
    return 1;
}
```

带表头结点的单链表

- 表头结点位于表的最前端，本身不带数据，仅标示表头
  - 作用1：统一空表与非空表的操作，简化链表操作的实现
  - 作用2：用于记录整个表的一些信息(如表长)

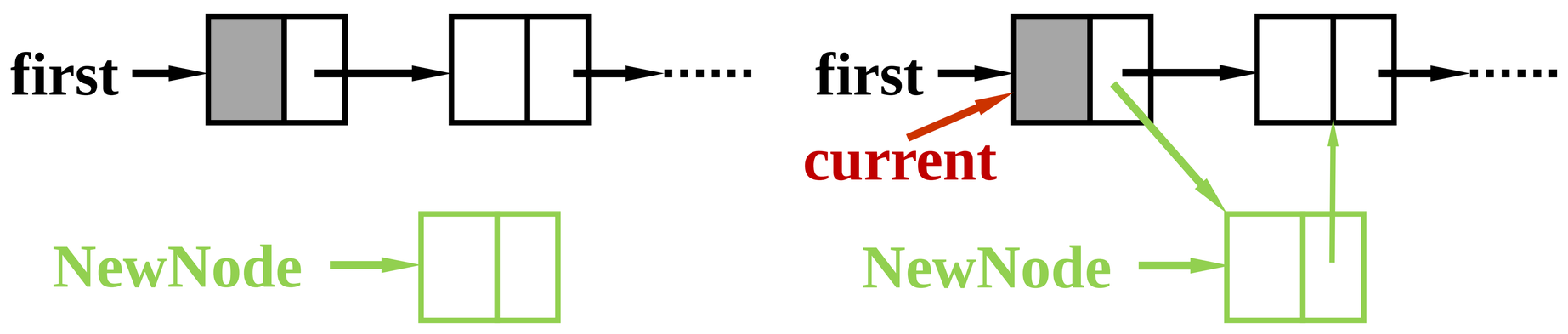


非空表



空表

在带表头结点的单链表最前端插入

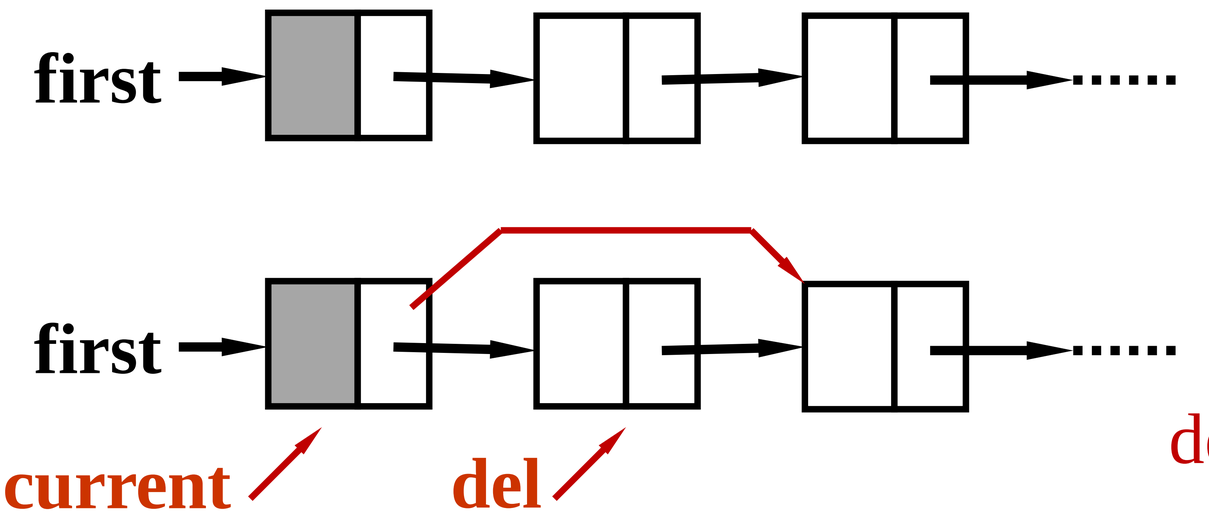




NewNode->link = current->link;

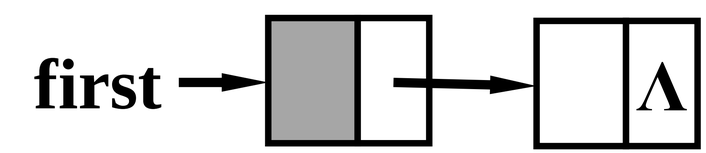
current->link = NewNode;

在带表头结点的单链表最前端删除

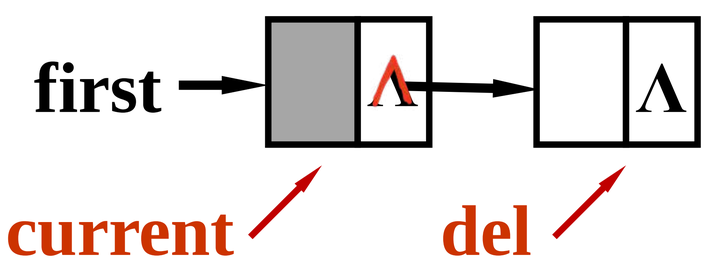


删除后非空表

```
del = current->link;
current->link = del->link;
delete del;
```

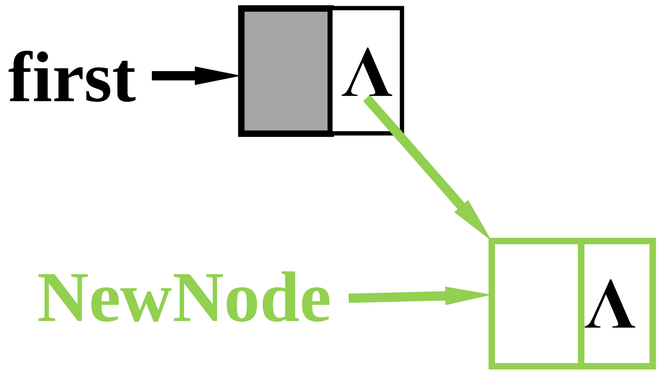


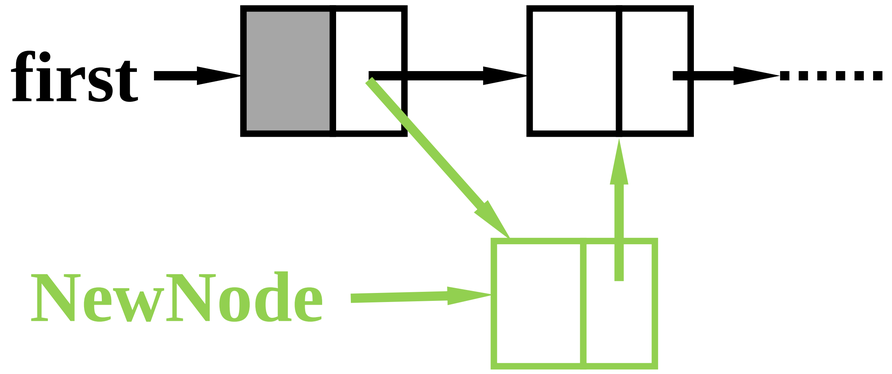
删除后空表



建立单链表(新结点从表头加入)

- 从空表开始，重复读入数据(以带表头结点的链表为例)：
  - 生成新结点
  - 将读入数据存放到新结点的数据域中
  - 将该新结点插入到链表的前端
  - 直到读入结束符为止





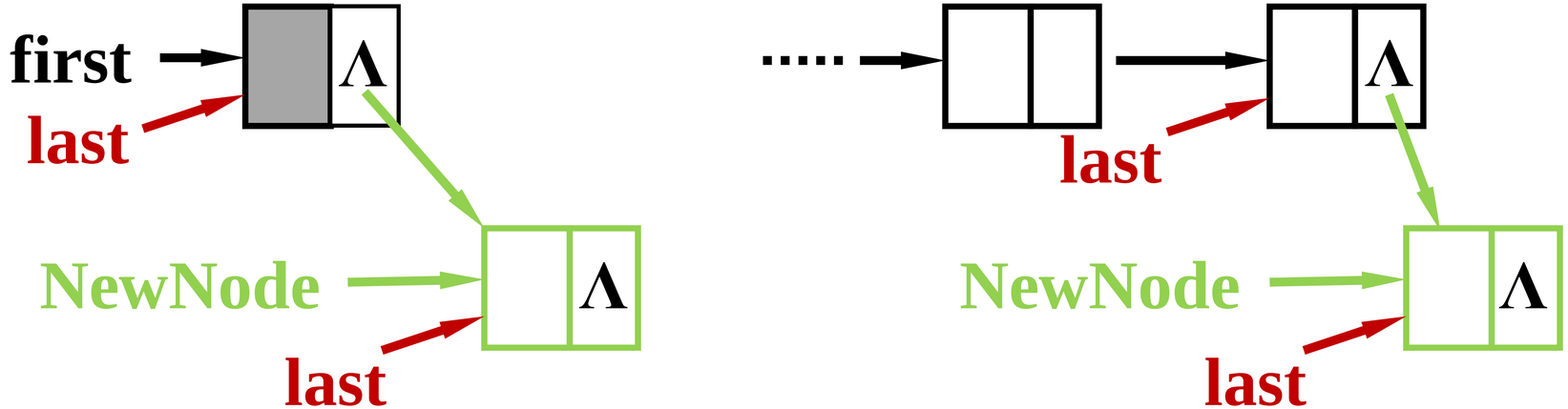

```
NewNode->link = first->link;
first->link = NewNode;
```

建立单链表(新结点从表头加入)

```
template <class Type> void List <Type> :: CreateListF(Type endTag) {
    Type val;
    LinkNode <Type> * NewNode;
    first = new LinkNode <Type>;  // 表头结点
    cin>>val;
    while ( val != endTag) {
        NewNode = new LinkNode <Type> (val);
        NewNode->link = first->link;
        first->link = NewNode;  // 插入到表前端
        cin>>val;
    }
}
```

建立单链表(新结点从表尾加入)

- 从空表开始，重复读入数据(以带表头结点的链表为例)：
  - 每次将新结点插到链表的表尾
  - 设置尾指针last，指向表中最后一个结点，新结点插在它后面
  - 尾指针last初始时置为指向表头结点地址



每次更新尾指针

last->link = NewNode;

last = NewNode;

建立单链表(新结点从表尾加入)

```
template <class Type> void List <Type> :: CreateListR(Type endTag) {
    Type val;
    LinkNode <Type> * NewNode, * last;
    first = new LinkNode <Type>;  // 表头结点
    last = first;
    cin>>val;
    while ( val != endTag ) {
        NewNode = new LinkNode <Type> (val);
        last->link = NewNode;  // 插入到表末端
        last = NewNode;  // last指向表尾
        cin>>val;
    }
}
```

练习1

- 求顺序表和单链表的长度的渐进时间复杂度
  - 顺序表的长度可以直接从最后一个元素的位置获得，而单链表的长度则必须遍历整个表后获得，即O(1)与O(n)

- 给定n个元素，求建立一个有序单链表的渐进时间复杂度
  - 单纯建立单链表的渐进时间复杂度为O(n)，而建立一个有序单链表还涉及查找正确的插入位置，故渐进时间复杂度为$O(n^2)$

比较次数：$0+1+2+\cdots+(n-1)$

- 对顺序表和单链表的逆置运算的最好渐进时间复杂度
  - 进行逆置运算时，顺序表的时间要少于单链表，量级均为O(n)

逆置顺序表：第一个和最后一个，第二个和倒数第二个……进行互换

逆置单链表：遍历一遍单链表，每取一个元素下来，把它插入在表头的位置

多项式的链表表示

- 在多项式的链表表示中每个结点三个数据成员

Data ≡ Term



- 优点
  - 多项式的项数可以动态地增长，不存在存储溢出问题
  - 插入、删除方便，不移动元素

多项式链表类的定义

```
class PolyLinkedList;  // 多项式链表类的前向引用声明
class TermNode {  // 多项式链表中项结点的类定义
    friend PolyLinkedList;  // 定义多项式链表类为友元类
    public:
        TermNode( ) { link = NULL; }  // 构造函数
    private:
        float coef;  // 系数
        int exp;  // 指数
        TermNode * link;  // 后继指针
};
```

多项式链表类的定义

```
class PolyLinkedList {  // 多项式链表类定义
    public:
        // 建立系数为c指数为e的新结点，并插入至pc指针所指结点的后面
        TermNode * InsertTerm(TermNode *pc, int c, int e);
        // 求以ah与bh为头结点的两个多项式链表的相加
        TermNode * PolyAdd(TermNode *ah, TermNode *bh);
        ……
};
```

P为搜索指针
h为头指针

多项式链表的相加

```
// 求以ah与bh为头结点的两个多项式链表的相加
TermNode * PolyLinkedList :: PolyAdd(TermNode *ah, TermNode *bh) {
    TermNode * pa, * pb, * pc, * ch;
    ch = new TermNode(NULL);  // 建立结果多项式链表的头结点
    pc = ch;  // 以ch为头结点的结果多项式链表检测指针
    pa = ah;  // 以ah为头结点的多项式链表检测指针
    pb = bh;  // 以bh为头结点的多项式链表检测指针
    while ( pa != NULL && pb! = NULL ) {
        if ( pa->exp == pb->exp ) {
            if ( pa->coef + pb->coef != 0 )  // 对应项系数相加非零
                // 插入系数为和值的新结点至结果多项式链表中
                pc = InsertTerm(pc, pa->coef + pb->coef, pa->exp);
            pa = pa->link; pb = pb->link;  // 检测指针后移
        }
```

辅助

多项式链表的相加

```
else if ( pa->exp < pb->exp) {
    pc = InsertTerm(pc, pa->coef, pa->exp);  pa = pa->link;
else {
    pc = InsertTerm(pc, pb->coef, pb->exp);  pb = pb->link;
}
    }
    while ( pa != NULL ) {
        pc = InsertTerm(pc, pa->coef, pa->exp);  pa = pa->link;
    }
    while (pb!=NULL) {
        pc = InsertTerm(pc, pb->coef, pb->exp);  pb=pb->link;
    }
    pc->link = NULL;  // 结果多项式链表收尾
    pc = ch; ch = ch->link; delete pc;  // 释放最初设置的辅助结点
    return ch;
}
```

多项式链表的相加

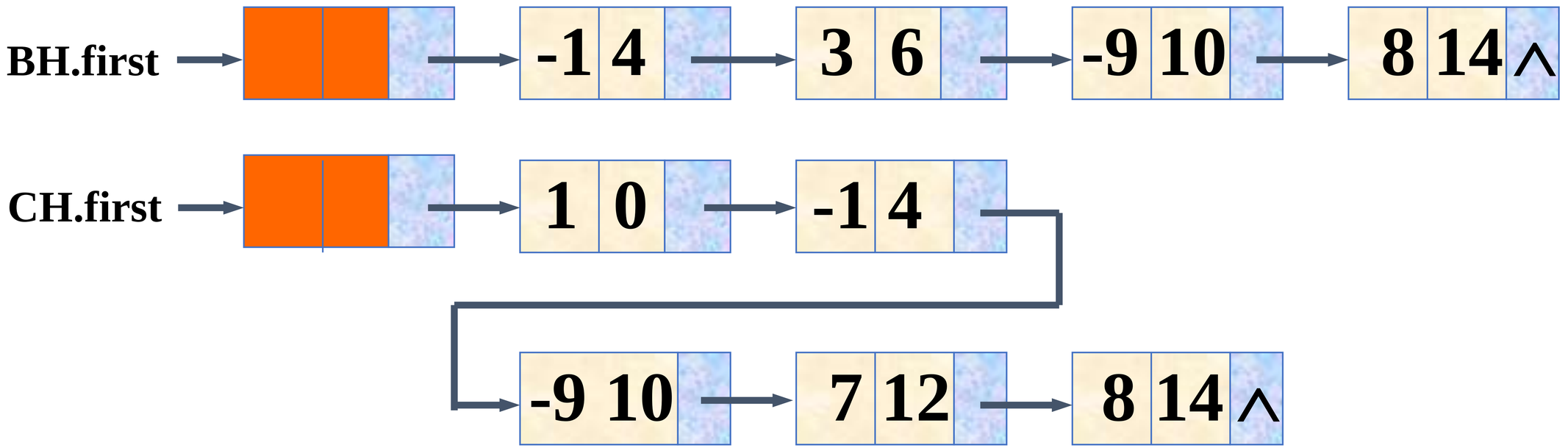
```
TermNode * PolyLinkedList :: InsertTerm(TermNode *pc, int c, int e) {
    TermNode * t = new <TermNode> (NULL);
    t->coef = c;  t->exp = e;
    pc->link = t;
    return t;
}
```

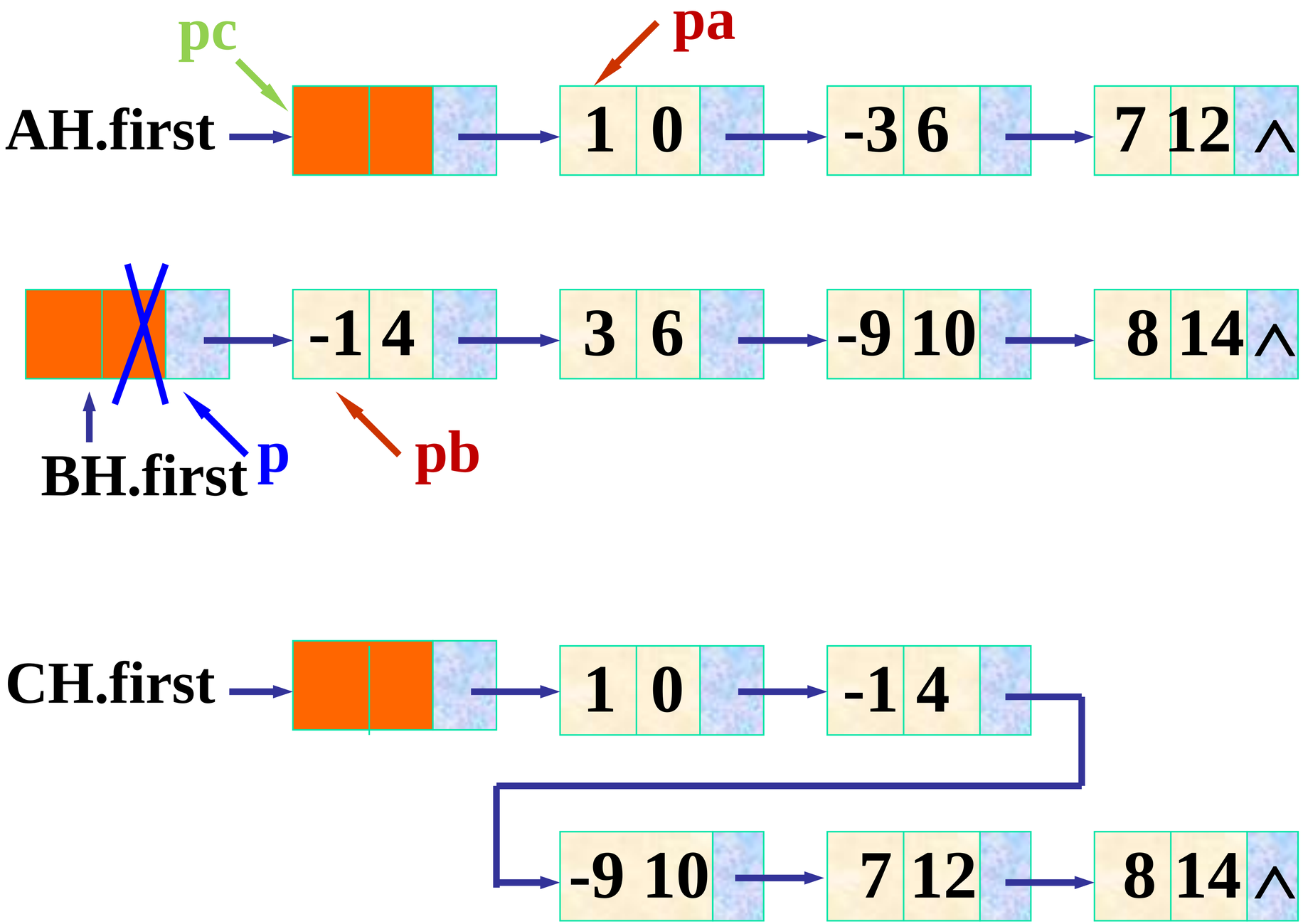
多项式链表的相加

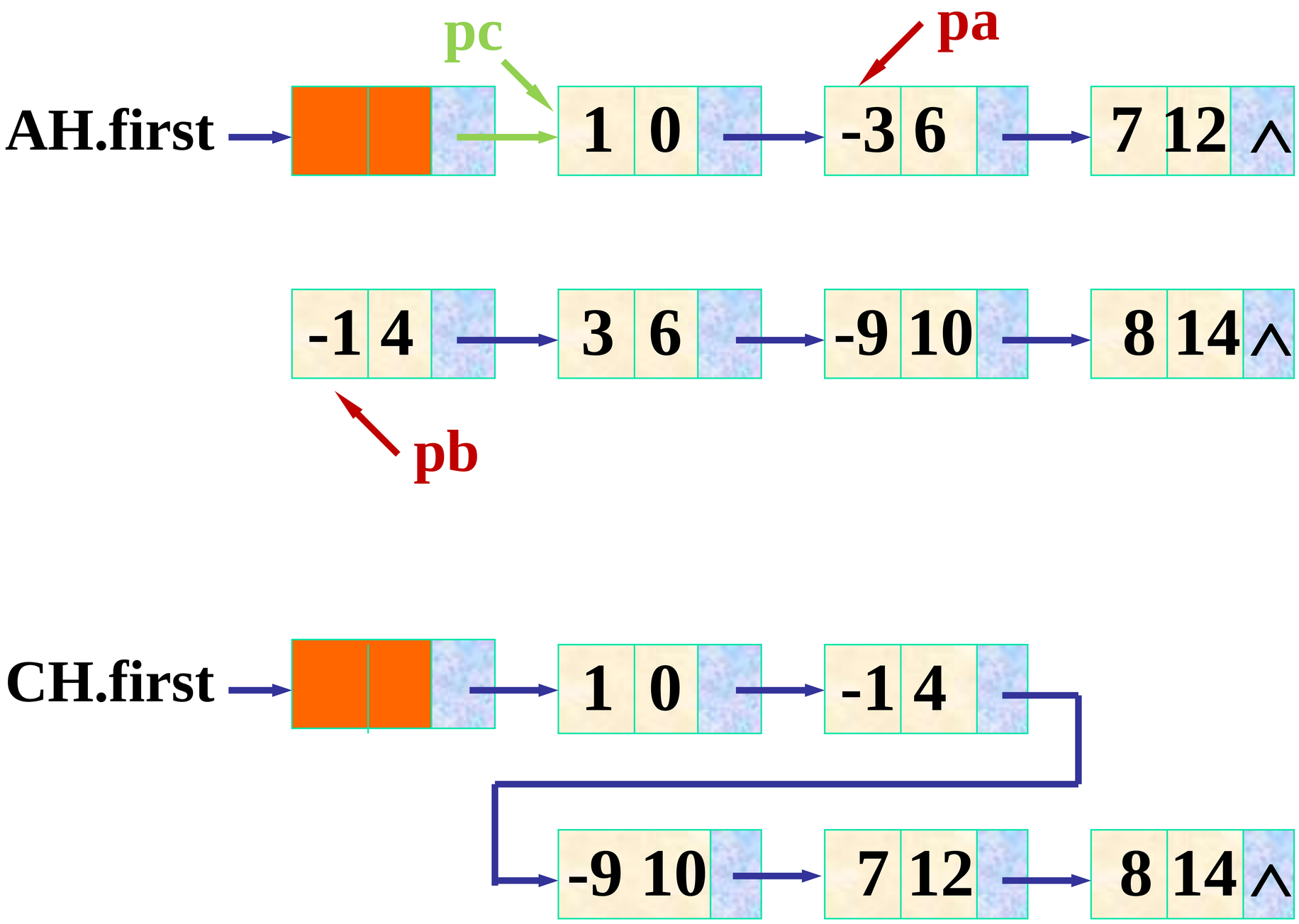
- **不保留原多项式，利用原多项式已有空间**

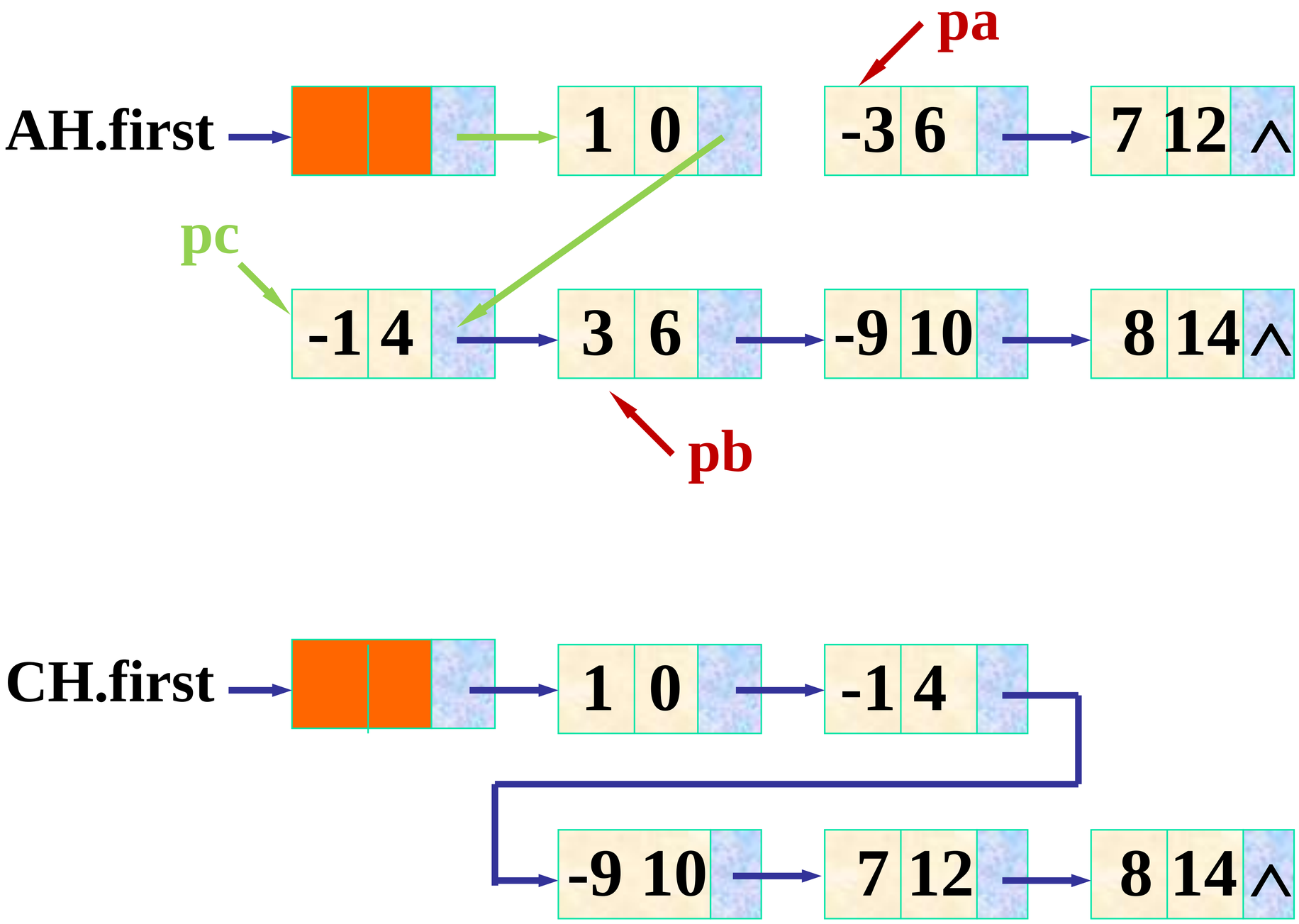
$$AH=1-3x^6+7x^{12},\ BH=-x^4+3x^6-9x^{10}+8x^{14}$$



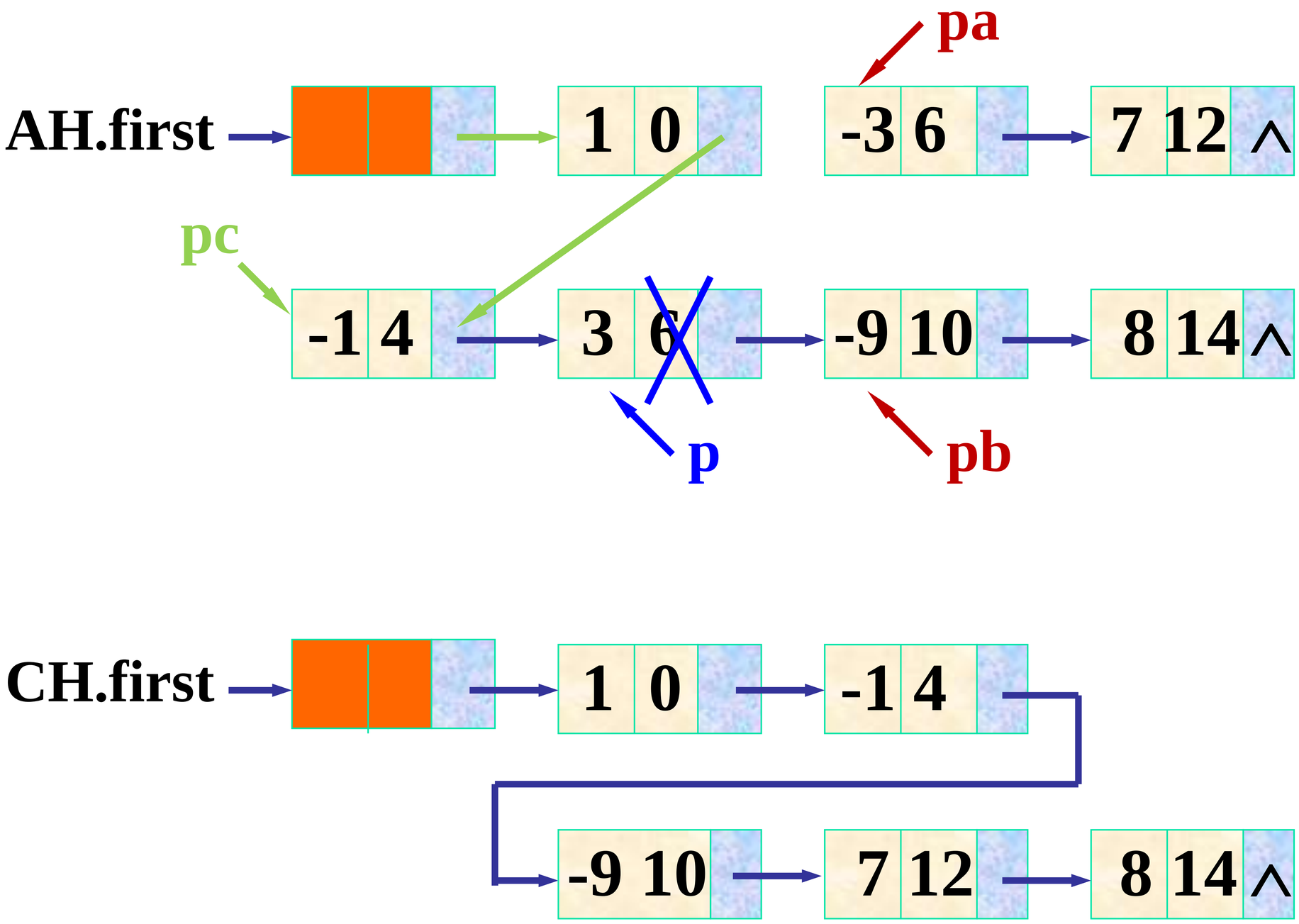


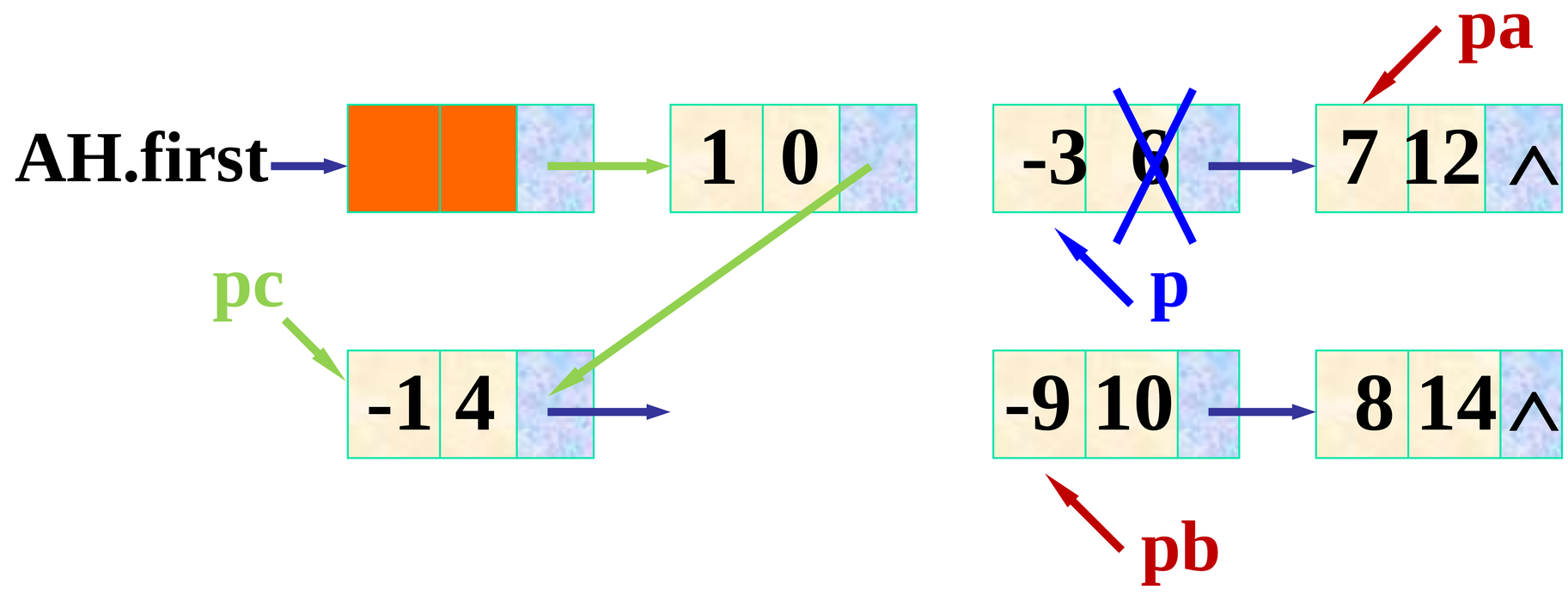

pc
pa
AH.first
1 0
-3 6
7 12 ∧
-1 4
3 6
-9 10
8 14 ∧
BH.first
p
pb
CH.first
1 0
-1 4
-9 10
7 12
8 14 ∧

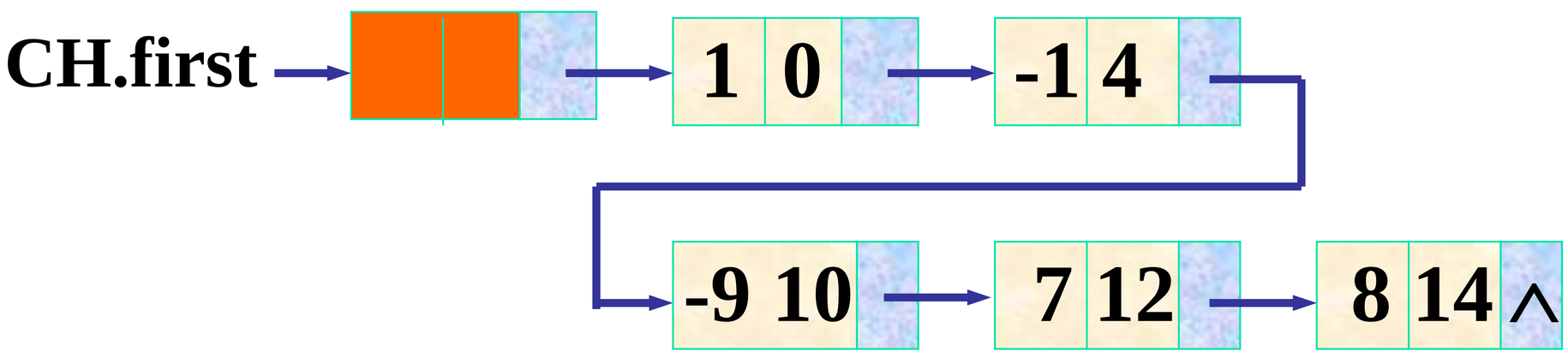

pc
pa
AH.first
1 0
-3 6
7 12 ∧
-1 4
3 6
-9 10
8 14 ∧
pb
CH.first
1 0
-1 4
-9 10
7 12
8 14 ∧

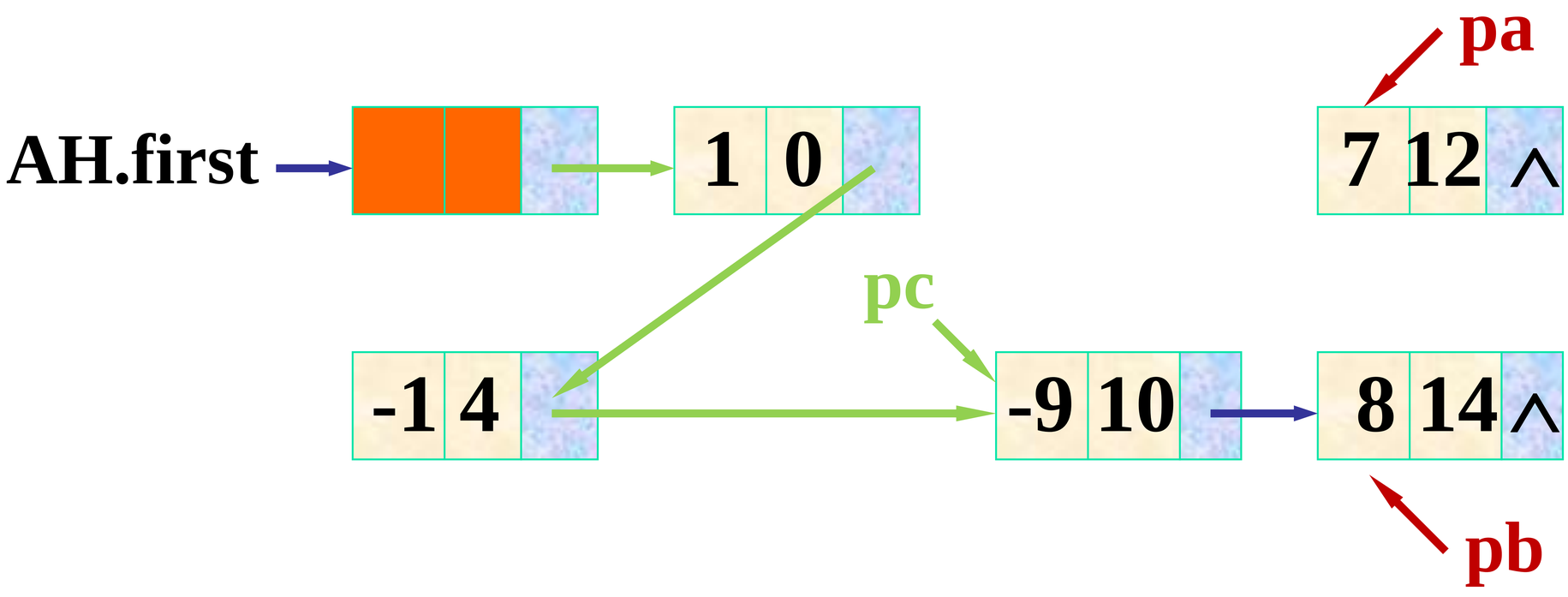

AH.first
1 0
-3 6
7 12 ∧
pa
pc
-1 4
3 6
-9 10
8 14 ∧
pb

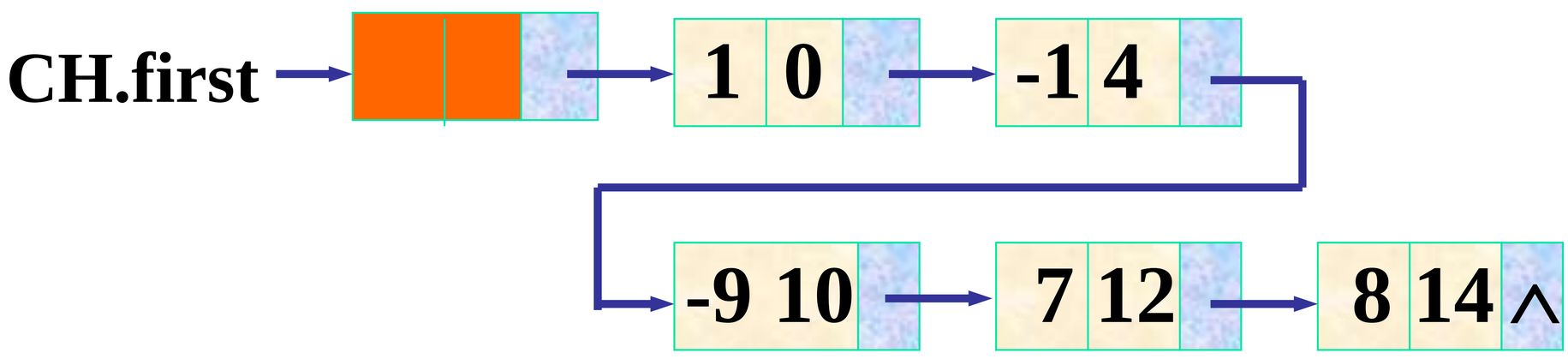
CH.first
1 0
-1 4
-9 10
7 12
8 14 ∧

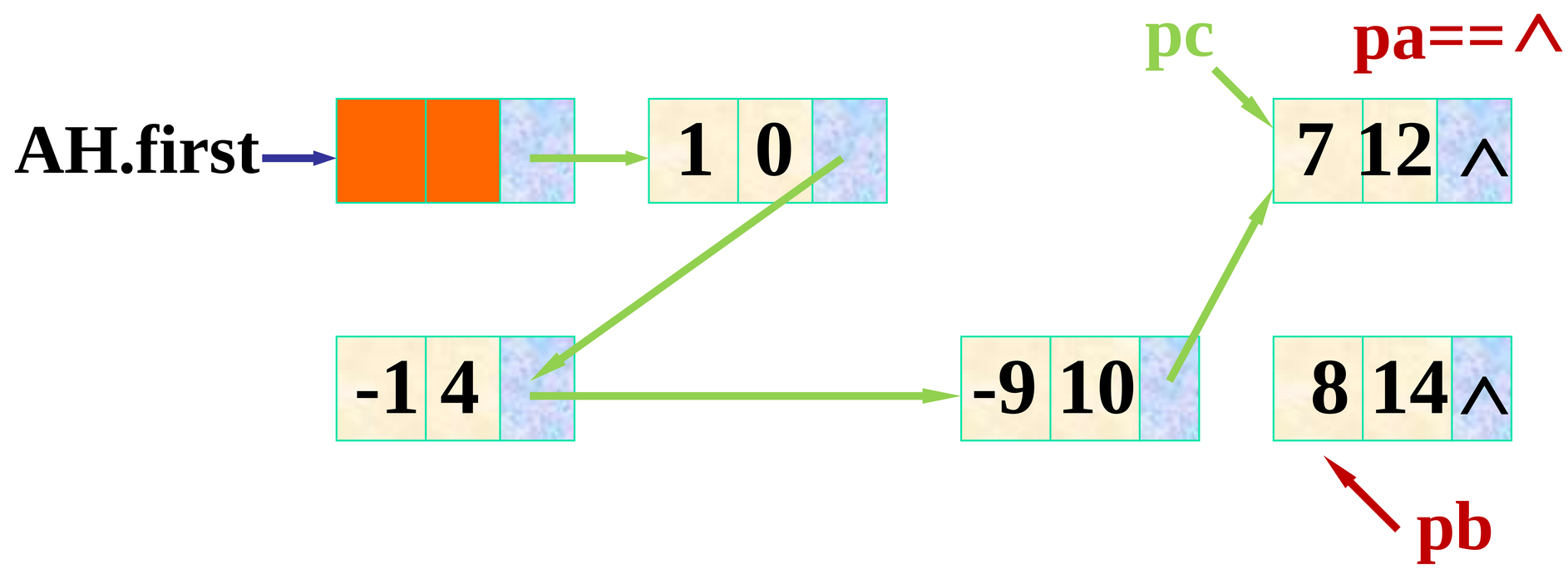

AH.first
1 0
-3 6
7 12 ^
pa
pc
-1 4
3 6
-9 10
8 14 ^
p
pb
CH.first
1 0
-1 4
-9 10
7 12
8 14 ^

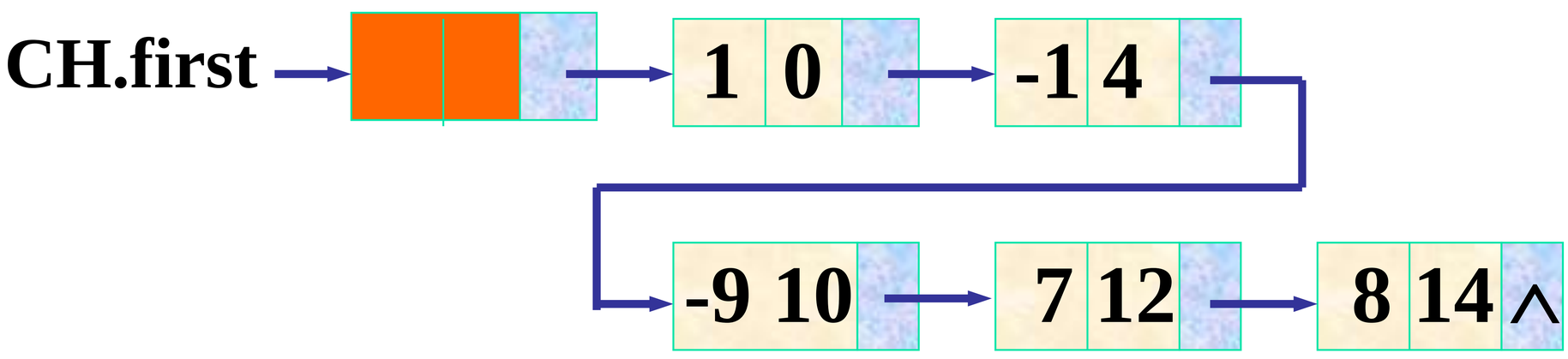

AH.first
1 0
pc
-1 4
pa
-3 6
7 12 ∧
p
-9 10
8 14 ∧
pb

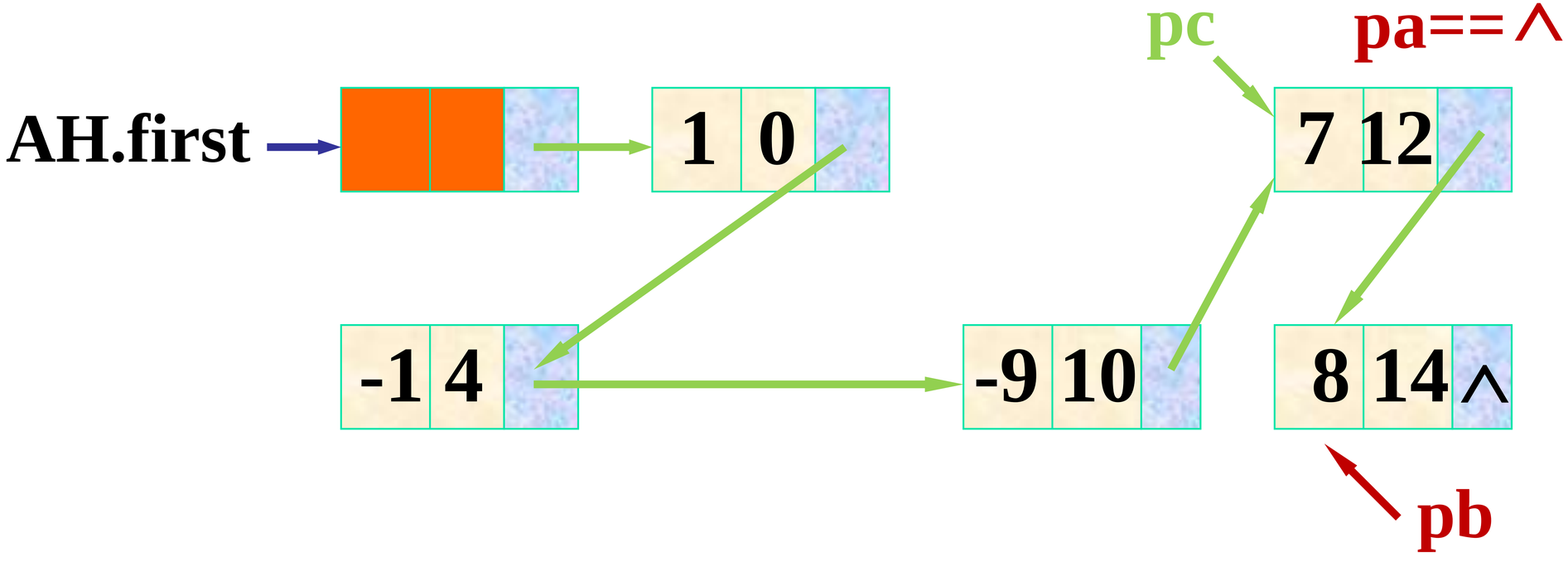

CH.first
1 0
-1 4
-9 10
7 12
8 14 ∧

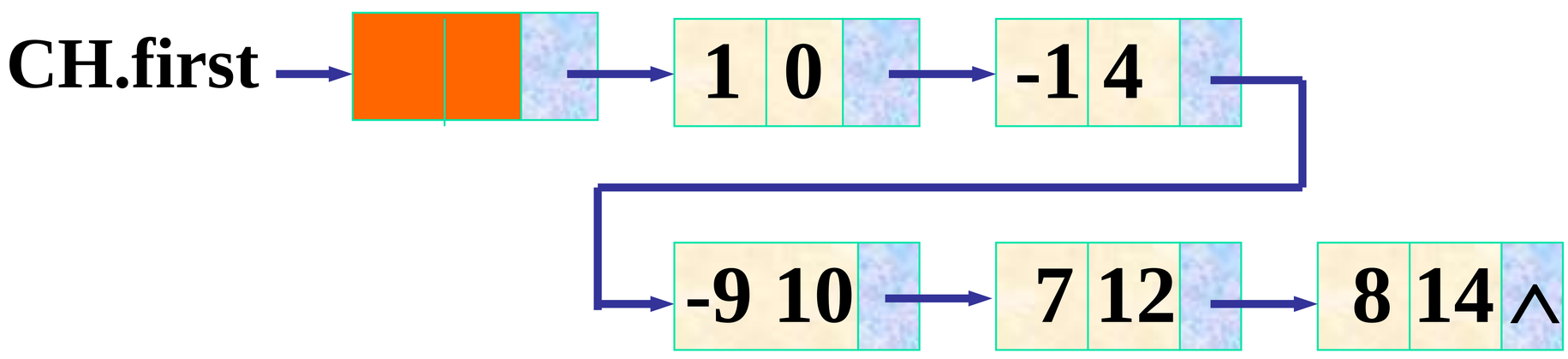

AH.first
1 0
pa
7 12 ∧
pc
-1 4
-9 10
8 14 ∧
pb


CH.first
1 0
-1 4
-9 10
7 12
8 14 ∧


AH.first
1 0
-1 4
-9 10
7 12 ∧
8 14 ∧
pc
pa==∧
pb


CH.first
1 0
-1 4
-9 10
7 12
8 14 ∧


AH.first
1 0
-1 4
-9 10
7 12
8 14 ^
pc
pa== ^
pb


CH.first
1 0
-1 4
-9 10
7 12
8 14 ^

链表的游标类(Iterator)

- 在单链表上经常需要执行的操作包括
  - 打印一个单链表中所有数据的值
  - 当数据为整数、浮点数或双精度数时，计算所有数据的累加和、乘积，或者求它们最大值、最小值、中值、平均值、方差等
  - 检索链表中所有满足某种条件的数据、以及对它们进行排序
- 为实现上述操作，都需要前后搜索整个链表
  - 若将所有操作都加入到链表类中，会使成员函数集变得很大
  - 新需求会不断地修改类定义，不利于建立类库以及类的复用

链表的游标类(Iterator)

- 链表游标类(Iterator)主要用于遍历链表的全部元素

- 链表游标类的定义原则
  - Iterator类是List类和ListNode类的友元类
  - Iterator对象引用已有的List类对象
  - Iterator类需记录对单链表最近处理到哪一个结点(current)
  - Iterator类提供若干检查和搜索操作

游标类的定义

```
template <class Type> class List;
template <class Type> class ListIterator;
template <class Type> class LinkNode {
    friend class List <Type>;
    friend class ListIterator <Type>;
    ……
    private:
        Type data;
        LinkNode <Type> *link;
};
template <class Type> class List {
    friend class ListIterator <Type>;
    ……
    private:
        LinkNode <Type> * first;
};
```

游标类的定义

```
template <class Type> class ListIterator {
    private:
        const List <Type> & list;  //引用已有链表
        LinkNode <Type> * current;  //当前结点指针
    public:
        // 构造函数: 引用链表l，表头为当前结点
        ListIterator(const List <Type> & l) : list( l ), current( l.first ) { }
        bool NotNull( );  // 检查当前结点是否非空
        bool NextNotNull( );  // 检查下一结点是否非空
        Type * First( );  // 返回第一个结点的值的地址
        Type * Next( );  // 返回下一个结点的值的地址
};
```

游标类成员函数的实现

```
// 检查当前结点是否非空
template <class Type> bool ListIterator <Type> :: NotNull( ) {
    if ( current != NULL) return true;
    else return false;
}

// 检查下一结点是否非空
template <class Type> bool ListIterator <Type> :: NextNotNull( ) {
    if ( current != NULL && current->link != NULL ) return true;
    else return false;
}
```

游标类成员函数的实现

```
// 返回链表中第一个结点的值的地址
template <class Type> Type * ListIterator <Type> :: First( ) {
    if ( list.first != NULL ) {
        current = list.first;
        return &current->data;
    } else {
        current = NULL;
        return NULL;
    }
}
```

游标类成员函数的实现

```
// 返回链表中下一个结点的值的地址
template <class Type> Type * ListIterator <Type> :: Next( ) {
    if ( current != NULL && current->link != NULL ) {
        current = current->link;
        return &current->data;
    } else {
        current = NULL;
        return NULL;
    }
}
```

利用游标类计算所有元素的和

```
int sum(const List <int> & l) {
    ListIterator <int> li (l);  // 定义游标对象，current指向list.first
    if ( !li.NotNull( ) )  return 0;  // 链表为空时返回
    int retval = li.First( );  // 第一个元素
    while ( li.NextNotNull( ) )  // 链表未扫描完
        retval += *li.Next( );  // 累加
    return retval;
}
```

02 线性表

- 链表
- 单链表
- **循环链表**
- 双向链表
- 静态链表

循环链表(Circular List)

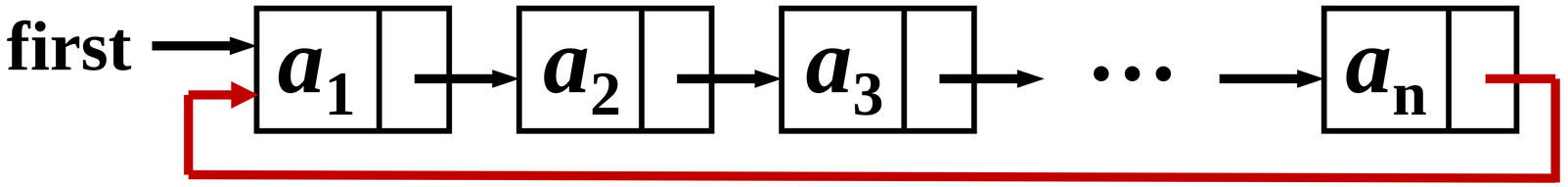
- 循环链表是单链表的变形，其最后一个结点的link指针不为NULL，而是指向表的前端(为简化操作，在循环链表中往往加入表头结点)

- 循环链表的特点是：只要知道表中任何一个结点的地址，就能遍历表中其它任一结点

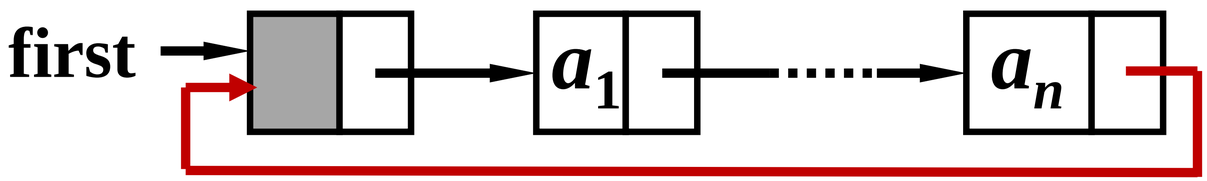
- 如何检查指针current是否到达链表的链尾？
  - 不是判断是否current->link == NULL
  - 而是判断是否current->link == first

循环链表示例

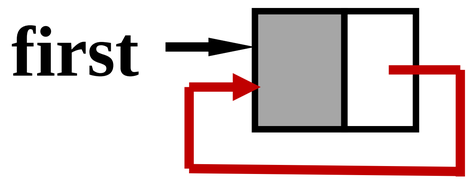
- 不带表头结点的循环链表



- 带表头结点的循环链表



非空表



空表

循环链表的类定义

```
template <class Type> class CircList;
template <class Type> class CircLinkNode {
    friend class CircList <Type>;
    public:
        // 构造函数
        CircLinkNode(Type d = 0, CircLinkNode <Type> * next = NULL);
    private:
        Type data;  // 结点数据
        CircLinkNode <Type> *link;  // 链接指针
};
```

循环链表的类定义

```
template <class Type> class CircList {
    private:
        // 循环链表的表头指针、尾指针
        CircLinkNode <Type> * first, * last;
    public:
        CircList(const Type &value);
        CircList(CircList <Type> &L);
        ~CircList( );
        int Length( ) const;
        CircLinkNode <Type> * GetHead( ) const;
        CircLinkNode <Type> * Search(Type value);
        ……
};
```

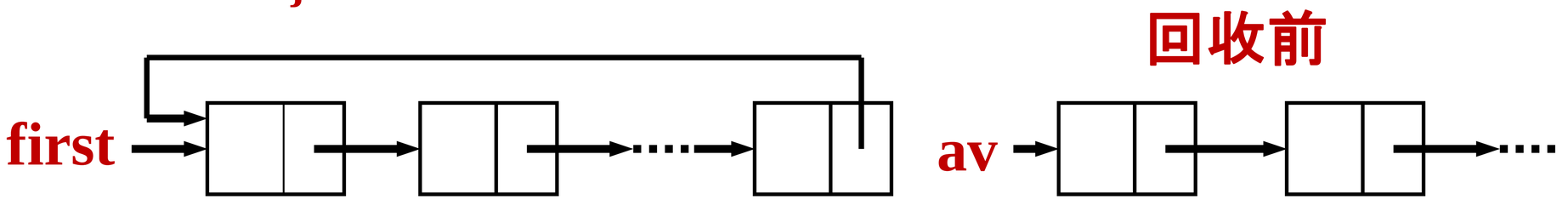
循环链表的搜索

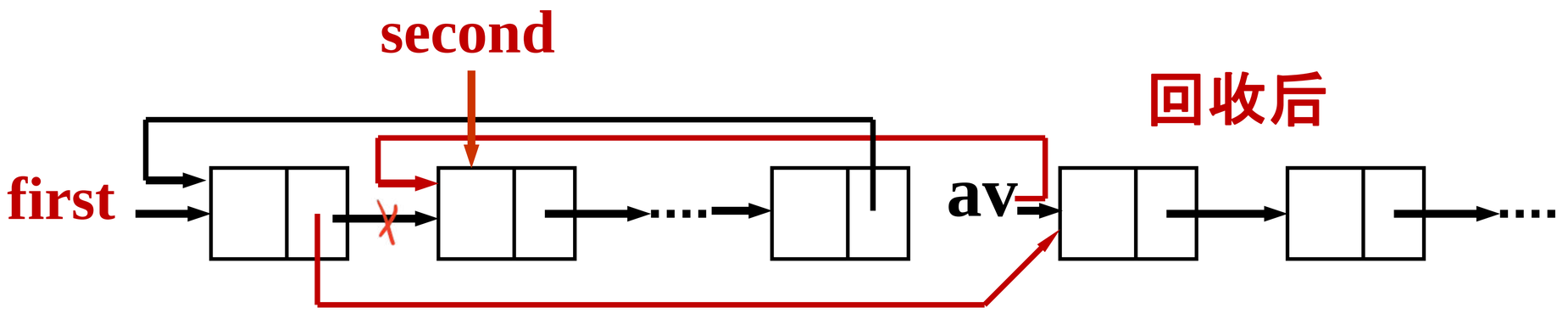
```
// 在链表中从头搜索其数据值为value的结点
template <class Type> CircLinkNode <Type>
                    * CircList <Type> :: Search (Type value) {
    CircLinkNode <Type> * current = first->link;
    while ( current != first && current->data != value)
        current = current->link;
    if ( current == first ) return NULL;
    else return current;
}
```

循环链表中不存在空指针

利用可利用空间表回收循环链表

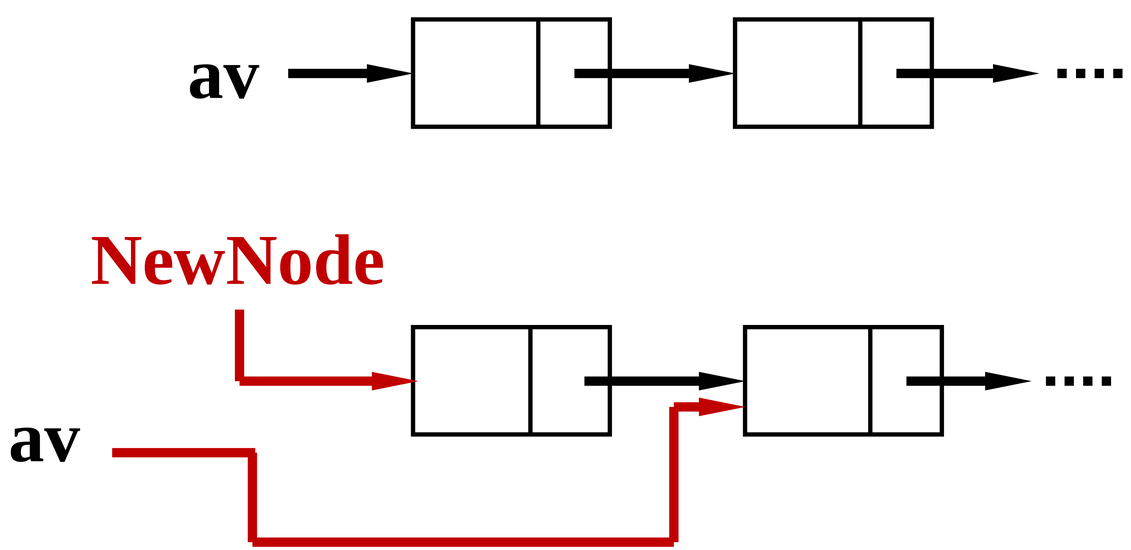
```
if ( first != NULL ) {
    CircLinkNode <Type> * second = first->link;
    first->link = av;
    av = second;
    first = second = NULL;
}
```



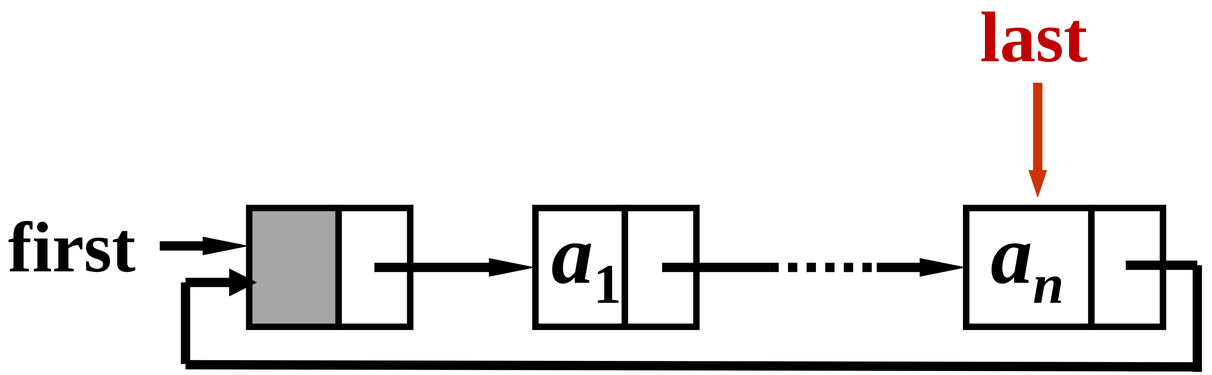


从可利用空间表分配结点

```
if ( av == NULL)
    NewNode = new CircLinkNode <Type>( );
else { NewNode = av; av = av->link; }
```



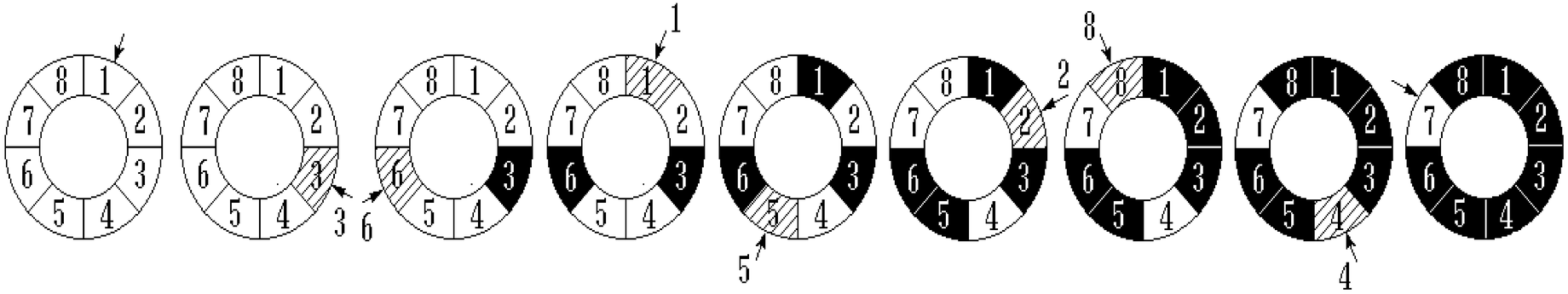
带尾指针的循环链表



- 若插入与删除仅在链表两端，可用带尾指针的循环链表
  - 在表尾插入，时间复杂性O(1)
  - 在表尾删除，时间复杂性O(n) 找到表尾前的一个元素
  - 在表头插入，相当于在表尾插入 O(1)
  - 在表头删除，时间复杂性O(1)

用循环链表求解约瑟夫问题

- 约瑟夫问题：n个人围成一个圈。首先，从第1个人开始，从1顺时针报数，报到第m个人，令其出列。然后，再从下一个人开始，从1顺时针报数，报到第m个人，令其出列。如此下去，直到圈中只剩一人为止，此人即为胜者



n = 8, m = 3

约瑟夫问题的解法

```
template <class Type> void Josephus (CircList <Type> & js, int n, int m) {
    CircLinkNode <Type> * p = js.GetHead( ), * pre = NULL;
    for ( int i = 1; i < n; i++ ) {  // 执行n-1次(出列n-1个人)
        for ( int j = 1; j < m; j++ ) {  // 找到第m个结点(移动m-1次)
            pre = p;  p = p->link;
        }
        cout<<“出列的人是”<<p->data<<endl;
        pre->link = p->link;  delete p;
        p = pre->link;
    }
}
```

练习2

- 非空的循环单链表first的链尾结点(由指针p所指向)满足什么条件？

  p->link = first;

- 设rear是指向非空的带表头结点的循环链表的链尾结点的指针，若想删除链表的第一个节点，应执行什么操作？

  del = rear->link->link;

  rear->link->link = del->link;

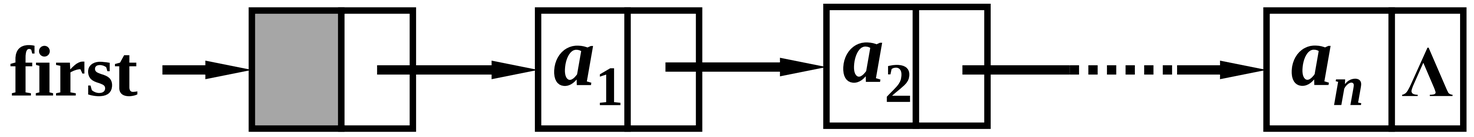
  delete del;

02 线性表

- 链表
- 单链表
- 循环链表
- **双向链表**
- 静态链表

单链表的问题？

- 在单链表中，搜索一个指定结点的后继结点非常方便
  - 只要该结点的link非空，就可通过link找到该结点的后继结点



- 在单链表中，搜索一个指定结点的前驱结点非常不容易
  - 必须从链头开始，沿link顺序检测，直到某一结点的后继结点为该指定结点，则此结点即为该指定结点的前驱结点

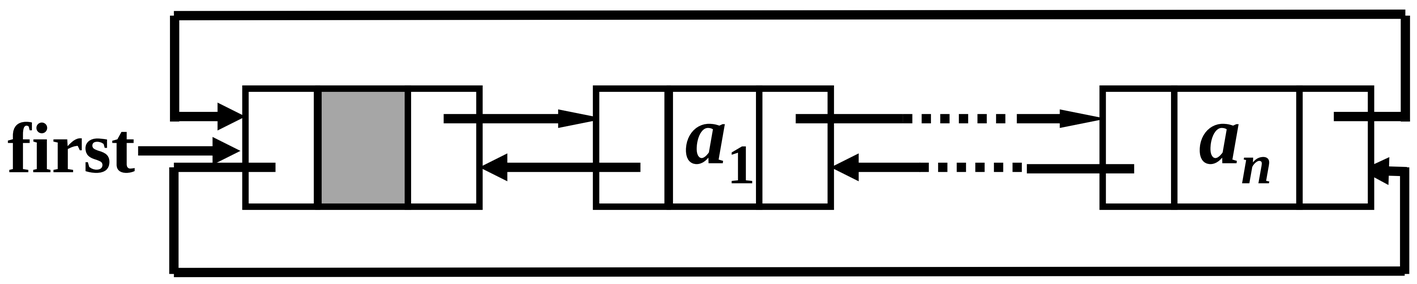
- 在一些问题中，经常要求检测指针向前驱和后继移动
  - 需要保证移动的时间复杂度达到最小

双向链表(Doubly Linked List)

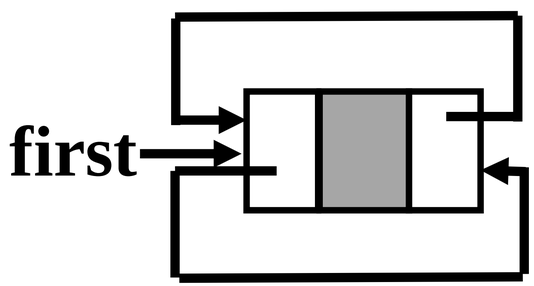
- **双向链表是指在前驱和后继方向都能遍历的线性链表，每个结点都有两个指针，一个指向结点的直接前驱，一个指向结点的直接后继**



- **双向链表通常采用带表头结点的循环链表形式**

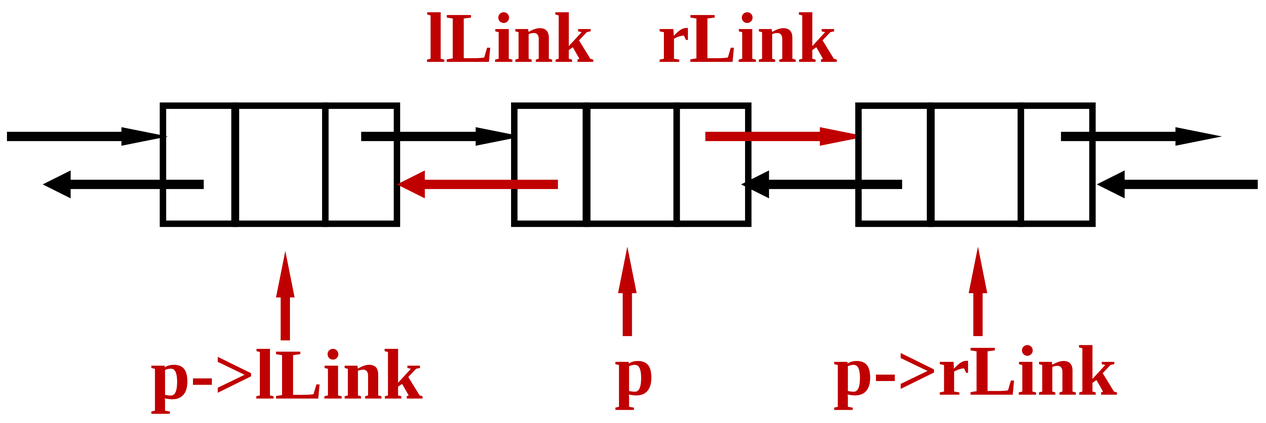


非空表



空表

双向链表中的结点指向



p = p->lLink->rLink = p->rLink->lLink

带表头结点的循环双向链表的类定义

```
template <class Type> class DblList;  // 循环双向链表类的前向引用声明
template <class Type> class DblNode {  // 循环双向链表结点类定义
    friend class DblList <Type>;
    private:
        Type data;  // 数据域
        DblNode <Type> * lLink, *rLink;  // 左指针域与右指针域
    public:
        DblNode(Type value) : data(value), lLink(NULL), rLink(NULL) { }
        DblNode(Type value, DblNode <Type> *left, DblNode <Type> *right) :
            data(value), lLink(left), rLink(right) { }
        Type GetData( ) { return data; }  // 获取结点数据域
};
```

带表头结点的循环双向链表的类定义

```
template <class Type> class DblList {  // 循环双向链表类定义
   public:
      DblList( );  // 构造函数
      DblList(Type uniqueVal);  // 构造函数：建立双向循环链表的表头结点
      DblList(DblList <Type> & RL);  // 复制构造函数：用已有链表RL初始化
      ~DblList( );  // 析构函数
      int Length( ) const;  // 计算循环双向链表的长度
      int IsEmpty( ) { return first->rLink == first; }
      int Find(const Type &x);  // 搜索数据域为给定值的结点
      // 建立数据域值为y的新结点，并插入至数据域值为x的结点后面
      int Insert(const Type &x, const Type &y);
      int Delete(const Type &x);  // 删除数据域值为x的结点
   private:
      DblNode <Type> * first;  // 表头指针
};
```

循环双向链表的构造函数

```
// 构造函数：建立双向循环链表的表头结点
template <class Type> DblList <Type> :: DblList(Type uniqueVal) {
    first = new DblNode <Type> ( uniqueVal);
    if ( first == NULL) { cerr <<" Memory Allocation Error\n ";  exit(1); }
    first->rLink = first->lLink = first;
}
```

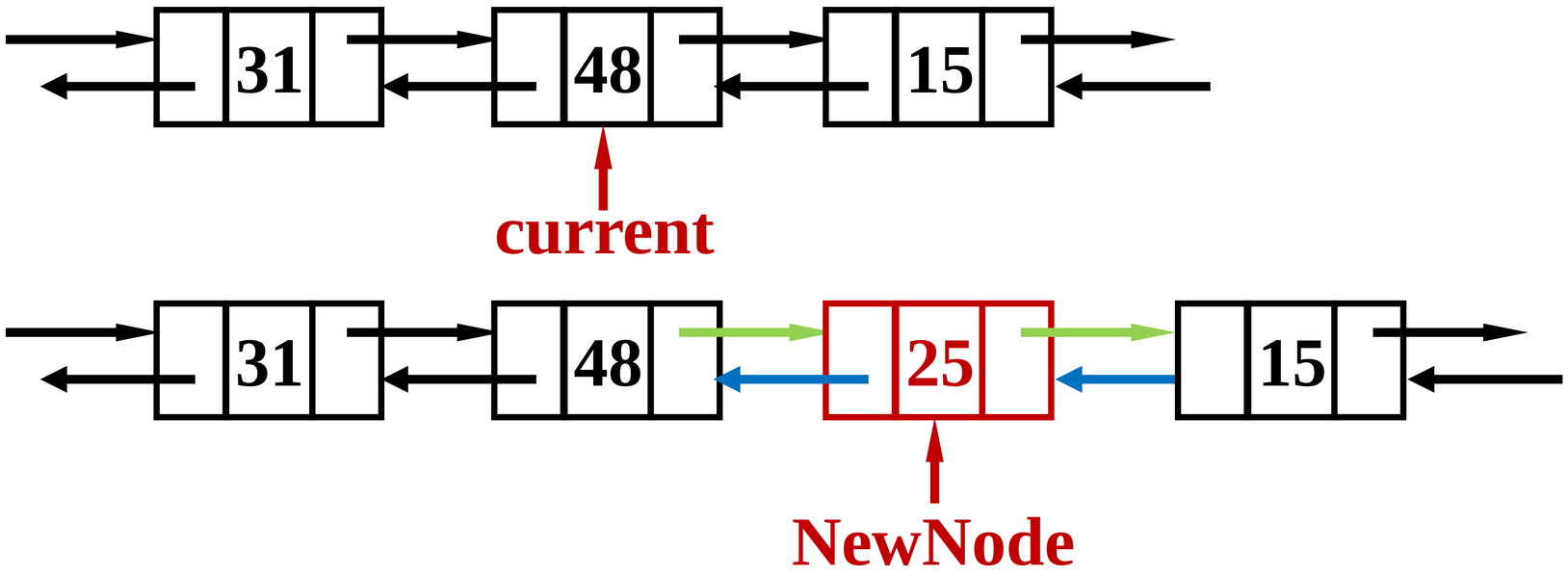
循环双向链表的复制构造函数

```
// 复制构造函数：用已有链表RL初始化
template <class Type> DblList <Type> :: DblList(DblList <Type> &RL) {
    first = new DblNode <Type> (RL->first->data);
    if ( first == NULL) { cerr<<" Memory Allocation Error\n "; exit(1); }
    first->lLink = first->rLink = first;                    last为尾指针
    LinkNode <Type> * p = RL->first->rLink, last = first;
    while ( p != RL->first ) {  // 逐个结点复制
        last->rLink = new DblNode <Type> (p->data, last, last->rLink);
        if ( last->rLink == NULL) { cerr<<" Memory Allocation Error\n "; exit(1); }
        last = last->rLink;
        last->rLink->lLink = last;  // 链接完成
        p = p->rLink;
    }
}
```

循环双向链表上的操作

- 有些操作与单循环链表相同，只需要用到一个方向的链
  - 求链表长度
  - 搜索链表
  - ……
- 插入、删除等操作则要调整2个方向的链

循环双向链表的插入



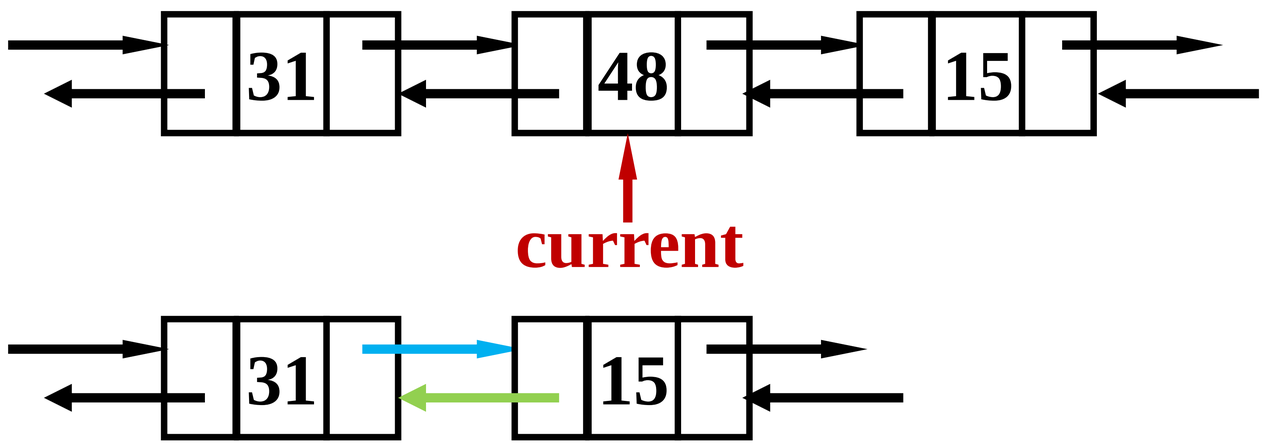
右
```
NewNode->rLink = current->rLink;
current->rLink = NewNode;
```

左
```
NewNode->lLink = current;
NewNode->rLink->lLink = NewNode;
```

循环双向链表的插入实现

```
// 建立数据域值为y的新结点，并插入至数据域值为x的结点后面
template <class Type> int DblList <Type> :: Insert(const Type &x, const Type &y) {
    DblNode * current, * NewNode;
    current = first->rLink;
    while ( current != first && current->data != x)  // 搜索数据域值为x的结点
        current = current->rlink;
    if ( current == first )  return 0;  // 搜索不成功，插入失败，返回0
    NodeNode = new DblNode (y, NULL, NULL);  // 建立数据域值为y的新结点
    NewNode->rLink = current->rLink;  // 插入新结点在适当位置并修改相应指针
    current->rLink = NewNode;
    NewNode->lLink = current;
    NewNode->rLink->lLink = NewNode;
    return 1;  // 插入成功，返回1
}
```

循环双向链表的删除



左 current->rLink->lLink = current->lLink;

右 current->lLink->rLink = current->rLink;

循环双向链表的删除实现

```
// 删除数据域值为x的结点
template <class Type> int DblList <Type> :: Delete(const Type &x) {
    DblNode * current = first->rLink;
    while ( current != first && current->data != x)  // 搜索数据域值为x的结点
        current = current->rLink;
    if ( current == first )  return 0;  // 搜索不成功，删除失败，返回0
    // 删除数据域值为x的结点并修改相应指针
    current->rLink->lLink = current->lLink;
    current->lLink->rLink = current->rLink;
    delete current;
    return 1;  // 删除成功，返回1
}
```

练习3

- **若想在不带表头结点的双向循环链表中的结点*p之后插入结点*s，应执行什么操作？**

A. p->rLink = s; s->lLink = p; p->rLink->lLink = s; s->rLink = p->rLink;

B. p->rLink = s; p->rLink->lLink = s; s->lLink = p; s->rLink = p->rLink;

C. s->lLink = p; s->rLink = p->rLink; p->rLink = s; p->rLink->lLink = s;

D. s->lLink = p; s->rLink = p->rLink; p->rLink->lLink = s; p->rLink = s;

练习3

- **判断一个带表头结点的双向循环链表是否对称相等的算法如下所示，请在空白处补充完整**

```
bool symmetry () {
    bool sym = true;
    DblNode * p = first->rLink;
    DblNode * q = first->lLink;
    while ( p != q && p->lLink != q &&  (1) )
        if ( p->data == q->data ) {
            (2)
            (3)
        }
        else sym = false;
    return sym;
}
```

单数次：相遇
双数次：错开

(1) sym == true

(2) p = p->rLink

(3) q = q->lLink

02 线性表

- 链表
- 单链表
- 循环链表
- 双向链表
- **静态链表**

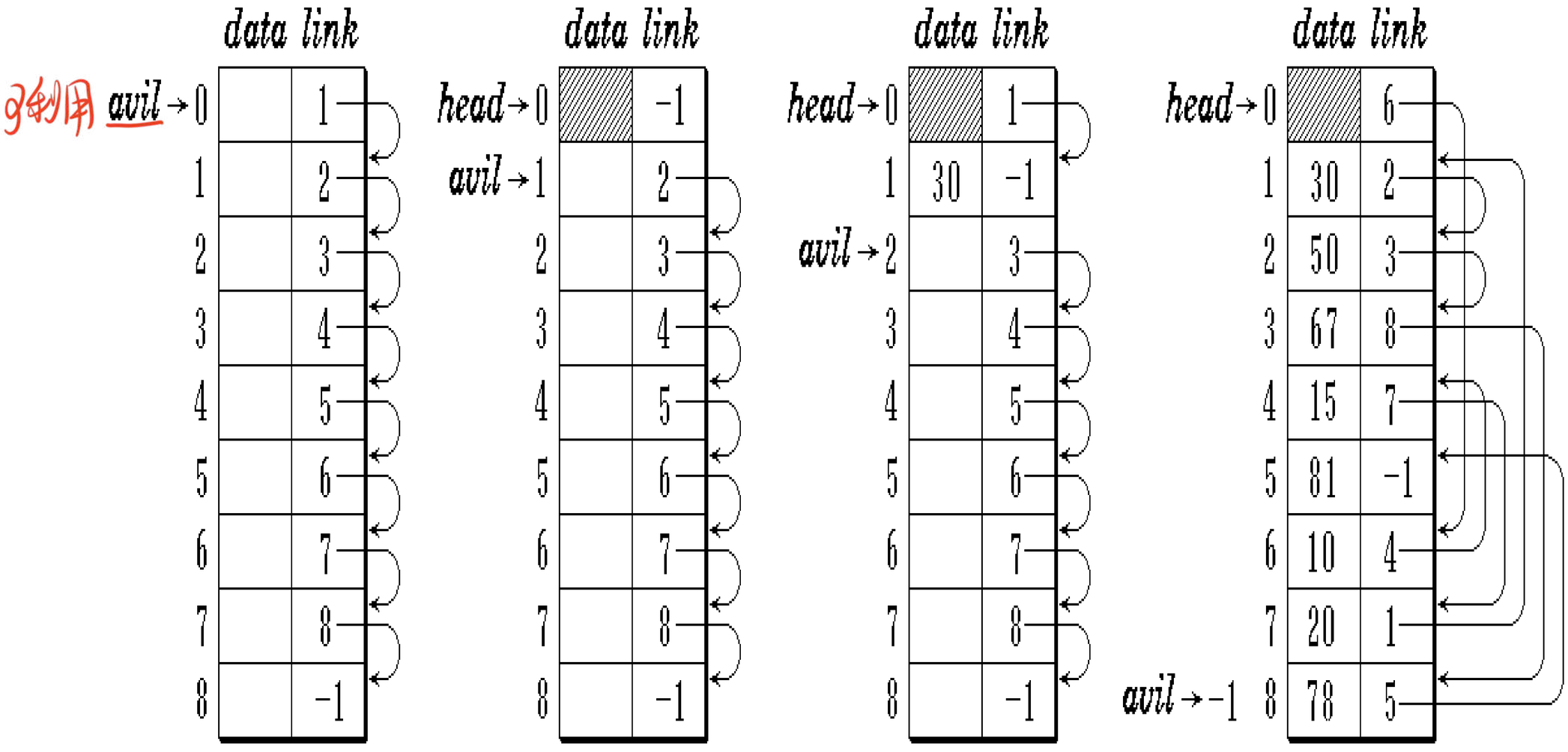
静态链表

- 可为不支持指针的高级语言提供链表功能
- 允许不改变各元素的物理位置，只要重新链接就能够改变这些元素的逻辑顺序

- 利用数组定义，在运算过程中存储空间大小不会变化
  - 每个结点由data域存储数据和link域存放链接指针
  - 所有结点形成一个结点数组
  - 可带有表头结点

-1相当于NULL

静态链表结构



未用空间　　链表初始化，创建头结点　　插入30，链入链表中　　经过若干加删后的链表状态

静态链表存储分配

- **需要分配一个结点时，从avil为头指针的链中摘下第一个结点分配出去，并用指针j指示**
- **头指针avil退到可利用空间表的第二个结点，即下一次可分配的结点地址**

j = avil; avil = elem[avil].link;

- **需要释放由指针i指示的结点时，将其链入avil所指示的链中的最前端，并由avil指示**

avil会连起了多个利用的空间

elem[i].link = avil; avil = i;

- **表头结点固定在第0个位置，其link指针指向链表中第一个结点，最后一个结点的link指针为-1，表示链表终止**

静态链表的类定义

```
const int maxSize = 100;  // 静态链表大小
template <class Type> class StaticList;
template <class Type> class SLinkNode {
    friend class StaticList <Type>;
    private:
        Type data;  // 结点数据
        int link;  // 结点链接指针
};
```

静态链表的类定义

```
template <class Type> class StaticList {
  private:
    SLinkNode <Type> elem[maxSize];
    int avil;  // 当前可分配空间首地址
  public:
    void InitList( );
    int Length( );
    bool IsEmpty( );
    int Search(Type x);
    int Locate(int i);
    bool Append(Type x);
    bool Insert(int i, Type x);
    bool Remove(int i);
};
```

初始化链表空间

```
template <class Type> void StaticList <Type> :: InitList( ) {
    elem[0].link = -1;  // 建立带表头结点的空链表
    avil = 1;  // 当前可分配空间从1开始
    for ( int i = 1; i < maxSize-1; i++)
        elem[i].link = i+1;  // 构成空闲链表
    elem[maxSize-1].link = -1;  // 空闲链表收尾
}
```

计算链表长度与判断链表空否

```
// 计算静态链表的长度
template <class Type> int StaticList <Type> :: Length( ) {
    int p = elem[0].link;  int i = 0;
    while ( p != -1 ) {
        p = elem[p].link;
        i++;
    }
    return i;
}
// 判断静态链表空否
template <class Type> bool StaticList <Type> :: IsEmpty( ) {
    if ( elem[0].link == -1)  return true;
    else return false;
}
```

在静态链表中查找具有给定值的结点

```
template <class Type> int StaticList <Type> :: Search(Type x) {
    int p = elem[0].link;  // 指针p指向链表第一个结点
    // 逐个结点检测查找具有给定值的结点
    while ( p != -1)
        if ( elem[p].data == x )  break;
        else p = elem[p].link;
    return p;
}
```

在静态链表中查找第i个结点

```
template <class Type> int StaticList <Type> :: Locate (int i) {
    if ( i < 0 )  return -1;  // 参数不合理
    if ( i == 0)  return 0;
    int j = 1, p = elem[0].link;
    // 循链查找第i号结点的位置
    while ( p != -1 && j < i) {
        p = elem[p].link;
        j++;
    }
    return p;
}
```

在静态链表的表尾追加一个新结点

```
template <class Type> bool StaticList <Type> :: Append(Type x) {
    if ( avil == -1 )  return false;  // 追加失败
    int q = avil;  // 分配结点
    avil = elem[avil].link;
    elem[q].data = x;
    elem[q].link=-1;
    int p = 0;  // 查找表尾
    while ( elem[p].link != -1 )  p = elem[p].link;
    elem[p].link = q;  // 追加
    return true;
}
```

在静态链表第i个结点处插入新结点

```
template <class Type> bool StaticList <Type> :: Insert(int i, Type x) {
    int p = Locate(i);
    if ( p == -1 )  return false;  // 找不到结点
    int q = avil;  // 分配结点
    avil = elem[avil].link;
    elem[q].data = x;
    elem[q].link = elem[p].link;  // 链入
    elem[p].link = q;
    return true;
}
```

在静态链表中删除第i个结点

```
template <class Type> bool StaticList <Type> :: Remove(int i) {
    int p = Locate(i-1);
    if ( p == -1) return false;  // 找不到结点
    int q = elem[p].link;  // 第i号结点
    elem[p].link = elem[q].link;
    elem[q].link = avil;  // 释放，链入可利用空间
    avil = q;
    return true;
}
```

本章小结

- 链表
- 单链表
- 循环链表
- 双向链表
- 静态链表

val= List[0]
for(int i=1; i< list.size(); i++){
if(val!=List(i))
remove(i)
}

作业

- 设有头结点的单链表，编程对表中任一值只保留一个结点，删除其余值相同的结点
- 编程实现“不保留原多项式，利用原多项式已有空间”的多项式链表相加操作
- 设ha和hb分别是两个带表头结点的非递减有序单链表的表头指针，编程实现将这两个有序链表合并成一个非递增的有序单链表，要求使用原来两个链表的存储空间，不另外占用其它存储空间
- 利用双向循环链表解决约瑟夫问题